ABSTRACT

Nowadays large no. of movies are published online in OTT's hence, and understanding whether a review is favourable or unfavourable supports recommendation, reputation monitoring and audience analysis. Reviews are difficult to classify because opinions are expressed through negation, irony and contrast that develop over long stretches of text. Because neural classifiers are much sensitive to small input changes and to hyper-parameter settings, both the embedding representation and the training configuration must be regularised and optimised.

The proposed system presents, a hybrid sentiment classifier a pre-trained DeBERTa-v3 encoder produces contextual token vectors. Then an Ordered-Neuron LSTM (ON-LSTM) decoder reads these vectors in sequence and organises its memory hierarchically through cumulative-softmax master gates, so that slowly changing neurons retains long-range polarity while fast neurons follow local phrasing. Later an attention-pooling layer condenses the sequence into one vector, and a fully connected layer with softmax gives the class probabilities. To improve robustness, the model is trained with the Fast Gradient Method (FGM), which perturbs the word-embedding matrix along the loss gradient. The main hyper-parameters are selected automatically with Optuna, using a Tree-structured Parzen Estimator sampler and a median pruner that stops the unpromising trials early.

The model was evaluated on the 50,000-review IMDB corpus with a stratified 70% 20% 10% split into 35,000 training, 5,000 validation and 10,000 test reviews. Hyper-parameters were turned using Optuna, running six trails of up to six epochs each on a held-out tuning subset where four trials ran to completion, and two were pruned, and the best configuration used 128 ON-LSTM units with chunk size 32, one layer and an adversarial strength of 0.820. Trained with this configuration, the final model reached 95.36 % accuracy, 95.31 % precision, 95.42 % recall and a 95.36 % F1-score on the test set, which indicates that the tuning did not overfit.

ACKNOWLEDGEMENT

First and foremost, we thank the **LORD ALMIGHTY** for his abundant blessings that is showered upon our past, present, and future successful endeavors.

We extend our sincere gratitude to our college management and Principal **Dr. S. Arivazhagan M.E, Ph.D.,** for providing a sufficient working environment, such as systems and library facilities. We also thank him very much for providing us with adequate lab facilities, which enable us to complete our project.

We would like to extend our heartfelt gratitude to **Dr. J. Raja Sekar M.E., Ph.D.,** Professor and Head, Department of Computer Science and Engineering, Mepco Schlenk Engineering College, for giving us the golden opportunity to undertake the project of this nature and for his most valuable guidance given at every phase of our work.

We would also like to extend our gratitude to **Dr.M.Jansi Rani B.E., M.Tech., Ph.D.,** Assistant Professor (Sl. Grade) Department of Computer Science and Engineering, for being our project coordinator and directing us throughout our project.

We would also like to extend our gratitude to **Mr. N. Umakanth M.E., Ph.D.,** Assistant Professor (Sl. Grade), Department of Computer Science and Engineering, for being our project guide and for his moral support and suggestions. He has put his valuable experience and expertise in directing, suggesting, and supporting us throughout the project to bring our best.

Our sincere thanks to our beloved **faculty members and lab technicians** for their help over our project work.

Finally, we extend our indebtedness towards our beloved **family and our friends** for their support.

TABLE OF CONTENTS

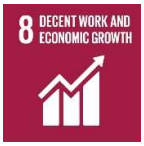
CHAPTER NO.	CONTENTS	PAGE NO.
	ABSTRACT	**iii**
	ACKNOWLEDGEMENT	**v**
	SDGs ALIGNMENT	**x**
	LIST OF TABLES	**xi**
	LIST OF FIGURES	**xii**
	LIST OF ABBREVIATIONS	**xiv**
1	**INTRODUCTION**	**1**
	1.1 Overview	1
	1.2 Problem Description	1
	1.3 Objectives of the Project	2
	1.4 Outcomes of the Project	3
	1.5 Organisation of the Project Report	3
	1.6 Summary	4
2	**LITERATURE SURVEY**	**5**
	2.1 Overview	5
	2.2 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	5
	2.3 DeBERTa: Decoding-enhanced BERT with Disentangled Attention	5
	2.4 DeBERTaV3: Improving DeBERTa using ELECTRA-style Pre-training with Gradient-Disentangled Embedding Sharing	6
	2.5 Long Short-Term Memory	6
	2.6 Ordered Neurons: Integrating Tree Structures into Recurrent Neural Networks ON-LSTM	6
	2.7 Adversarial Training Methods for Semi-supervised Text Classification	7

CHAPTER NO.	CONTENTS	PAGE NO.
	2.8 Algorithms for Hyper-Parameter Optimization	7
	2.9 Hyperband: A Novel Bandit-Based Approach to Hyper-Parameter Optimization	7
	2.10 Optuna: A Next-generation Hyperparameter Optimization Framework	8
	2.11 TRALSem: A Robust Model for Textual Sentiment Analysis	8
	2.12 Comparison Table of surveyed works	9
	2.13 Summary	9
3	**SYSTEM STUDY**	**14**
	3.1 Overview	14
	3.2 Existing Method	14
	3.2.1 Classical machine learning approaches	14
	3.2.2 Recurrent and convolutional neural approaches	14
	3.2.3 Transformer fine-tuning with single-vector pooling	14
	3.2.4 Limitations of the existing methods	15
	3.2.5 The closest existing system: TRALSem	15
	3.3 Proposed Method	15
	3.3.1 Text cleaning and encoding	16
	3.3.2 DeBERTa contextual encoding	16
	3.3.3 ON-LSTM decoding	16
	3.3.4 Attention pooling and classification	17
	3.3.5 FGM adversarial training	17
	3.3.6 Hyper-parameter optimisation with Optuna	17
	3.4 Comparison of Existing and Proposed Methods	17
	3.5 Summary	17
4	**SYSTEM DESIGN**	**19**
	4.1 Overview	19

CHAPTER NO.	CONTENTS	PAGE NO.
	4.2 System Architecture	19
	4.3 Dataset and Data Preprocessing	19
	4.3.1 Dataset	19
	4.3.2 Text cleaning	20
	4.3.3 Tokenisation and batching	20
	4.4 DeBERTa Encoder	21
	4.5 ON-LSTM Decoder	21
	4.6 Attention Pooling and Classifier	23
	4.7 Adversarial Training Module	24
	4.8 Hyper-parameter Search Module	24
	4.9 Summary	26
5	**SYSTEM IMPLEMENTATION**	**27**
	5.1 Overview	27
	5.2 Data Preprocessing and Tokenization	27
	5.3 Deberta Encoder	28
	5.4 ON-LSTM Decoder	28
	5.5 Attention Pooling and Classifier	29
	5.6 Adversarial Training Module	29
	5.7 Hyper-parameter Optimization Module	30
	5.8 Summary	31
6	**RESULTS AND DISCUSSION**	**32**
	6.1 Overview	32
	6.2 Dataset Description	32
	6.3 Optuna Hyper-parameter Search Results	32
	6.4 Convergence of the Optuna Search	34
	6.5 Final Model Results	36
	6.6 Error Analysis	37
	6.7 Comparison with the Existing System	38
	6.8 Discussion	41

CHAPTER NO.	CONTENTS	PAGE NO.
	6.9 Summary	41
7	**CONCLUSION AND FUTURE WORK**	**42**
	7.1 Conclusion	42
	7.2 Future Enhancements	42
	7.3 Social Impacts and Benefits	43
	7.4 Summary	43
	APPENDIX 1 – WORKING ENVIRONMENT	**44**
	APPENDIX 2 – SOURCE CODE	**45**
	REFERENCES	**58**

SDGs ALIGNMENT

SDG Icons	Relevance of SDG in the context of project	Section Name(s)	Page number(s)
	Documented, tested and reproducible code builds skills in natural language processing and machine learning among students and researchers	Objectives of the Project, Discussion, Social Impacts and Benefits.	2, 41, 43
	Sentiment analytics supports economic productivity and job opportunities in the film, media and digital-services industries	Social Impacts and Benefits.	43
	A novel hybrid DeBERTa–ON-LSTM architecture with adversarial training advances innovation and digital infrastructure for language technology	Problem Description, Objectives of the Project, Conclusion, Social Impacts and Benefits.	1, 2, 42, 43
	Automated hyper-parameter search with early pruning reduces wasted computing resources, and review analytics supports informed consumption	Problem Description, Objectives of the Project, Outcomes of the Project, Existing Method, Optimisation and Hyper-parameter Search Module, Hyper-parameter Optimisation Results, Social Impacts and Benefits.	1, 2, 3, 14, 24, 32, 43
	Use of open datasets and open-source libraries, and sharing of the implementation, promote knowledge sharing and partnership	Software Environment and Project Structure Social Impacts and Benefits.	27, 43

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparative summary of the surveyed works	15
3.1	Comparison between the existing and the proposed method	17
4.1	Stratified split of the IMDB corpus	20
5.1	Search space of the Optuna study	30
6.1	Optuna trials: validation F1-score after each epoch	33
6.2	Sampled parameters of the logged trials	33
6.3	Hyper-parameters of the final model	36
6.4	Epoch-wise results of the final training	36
6.5	Performance of the final model	37
6.6	Confusion matrix counts (reconstructed from the reported metrics)	37
6.7	TRALSem versus the proposed model: experimental set-up	38
6.8	IMDB test metrics: TRALSem versus the proposed model	39

LIST OF FIGURES

FIGURE NO.	FIGURE CAPTION	PAGE NO.
3.1	Block diagram of the existing Transformer fine-tuning approach	15
3.2	Workflow of the proposed model	16
4.1	System architecture of the proposed model	19
4.2	Data preprocessing and batching pipeline	21
4.3	Structure of the ON-LSTM cell	22
4.4	Attention-pooling mechanism	23
4.5	One training step with FGM adversarial training	24
4.6	Optuna hyper-parameter optimisation workflow	25
6.1	Class distribution of the training, validation and test sets	32
6.2	Validation F1-score of each Optuna trial after every epoch	33
6.3	Optimisation history (convergence) of the Optuna search	34
6.4	Computing time of the Optuna trials (trial 0: start time not logged)	35
6.5	Training and validation loss (left), validation F1-score and accuracy (right) of the final model	36
6.6	Confusion matrices of the final model on the validation and test sets	37
6.7	IMDB macro-F1 of the proposed model and of the systems reported in [25] (axis truncated at 0.78)	39
6.8	Precision, recall and macro-F1 on IMDB: RFLSem and TRALSem as reported in [25] and the proposed model (axis truncated at 0.935)	40

FIGURE NO.	FIGURE CAPTION	PAGE NO.
6.9	IMDB macro-F1 of the TRALSem adversarial variants reported in [25] and of the proposed model with FGM (axis truncated at 0.93)	40

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AMP	Automatic Mixed Precision
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CLS	Classification token
CSV	Comma-Separated Values
cumax	Cumulative softmax
DeBERTa	Decoding-enhanced BERT with disentangled attention
FC	Fully Connected
FGM	Fast Gradient Method
FN / FP	False Negative / False Positive
GELU	Gaussian Error Linear Unit
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
IMDB	Internet Movie Database
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
NFKC	Normalisation Form KC (compatibility composition)
NLP	Natural Language Processing
ON-LSTM	Ordered-Neuron Long Short-Term Memory
SDG	Sustainable Development Goal
TN / TP	True Negative / True Positive
TPE	Tree-structured Parzen Estimator
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In this chapter, problem description classifying sentiment in user-written movie reviews and explains how hard it is to classify them, the purpose, objectives, outcomes, organization of the report and the applications of the project are discussed which gives overview of the entire project work.

1.2 PROBLEM DESCRIPTION

It is challenging to detect sentiment from movie reviews by the users because often opinions are negated, expressesed with sarcasm or contrast and also these are dispersed across lengthy sections of text. Reading every review manually is impossible, so the field of sentiment analysis tries to decide automatically whether a piece of text is favourable or unfavourable. Because this analytics capability allows organisations to turn unstructured text into decisions, it contributes to technological innovation and to the digital infrastructure of the entertainment industry (SDG 9).

Although the task is a binary decision, it is not simple. A reviewer may praise the acting and still criticise the film, may use irony such as “a smile on his face, but he is not actually good with the movie”, or may negate a positive expression several words after it appears. Early statistical classifiers based on word counts cannot represent such dependencies [1], and models that read text only in a single fixed way often fail on long reviews in which the opinion changes from one clause to the next [2].

The Transformer encoders such as BERT and DeBERTa have raised the quality of text representations considerably [6, 8]. However, in the most common fine-tuning set-up the whole review is summarised by the representation of a single special token, and the model does not explicitly track how the sentiment develops as the text is read. In addition, neural classifiers are much sensitive to small changes in their input representation, that are difficult to fine-tune, and their performance depends strongly on hyper-parameters such as learning rates, dropout and hidden sizes [3]. Tuning these

values manually is slow and consumes a large amount of computing time, which motivates an automated and resource-efficient search procedure (SDG 12).

The proposed system proposes, a hybrid sentiment classifier. A DeBERTa-v3 encoder produces contextual token vectors, an Ordered-Neuron LSTM (ON-LSTM) decoder reads these vectors sequentially while organising its memory hierarchically, an attention-pooling layer forms a single review vector, and a fully connected network with softmax gives the final class probabilities. To improve robustness the model is trained with the Fast Gradient Method (FGM) applied to the word embeddings, and the main hyper-parameters are selected automatically with the Optuna framework using a Tree-structured Parzen Estimator (TPE) sampler and a median pruner.

1.3 OBJECTIVES OF THE PROJECT

- To design a data encoding module that converts raw text into content and positional vectors and feed it to Transformer.
- To construct a 12-layer transformer encoder with disentangled multi-head attention mechanism to capture long semantic relationships and provide Adversarial Training
- To develop a ON-LSTM layer to get Hierarchical representation of tokens
- To design an attention pooling module that takes important information from neuron and classifying the sentiment scores after Fullyconnected layer.
- To select the important hyper-parameters automatically with Optuna (TPE sampling and median pruning) while tuning model parameters, improving overall performance and keeping the computational effort low by stopping unpromising trials early (SDG 12).

1.4 OUTCOMES OF THE PROJECT

- The raw text is successfully transformed into robust numerical embedding that are resistance to adversarial inputs.
- Deep and long contextual relationships between words are effectively captured with the help of Transformer Encoder.
- Hierarchical representation of neurons are generated from low-level to high-level from which important information are taken with Attention weight.

- Attention pooling extracts important features and generates sentiment scores with the help of FC and softmax.
- Optimal hyper-parameter configurations are automatically identified through optuna search, resulting in improved model accuracy and reduced manual tuning effort.

1.5 ORGANISATION OF THE PROJECT REPORT

Chapter 2 reviews the related work on sentiment classification, pretrained encoders, recurrent models with structural bias, adversarial training and hyper-parameter optimisation, and discusses the advantages and limitations of each.

Chapter 3 presents the system study. It describes the existing approaches, their limitations and the proposed method at a conceptual level.

Chapter 4 describes the system design: the architecture, preprocessing, the encoder, the ON-LSTM decoder, the attention pooling, the adversarial module and the optimisation module, together with the mathematical formulation.

Chapter 5 explains the implementation, including the software modules, training engine, tuning script and tests.

Chapter 6 reports and discusses the results of the hyper-parameter search and of the final model on the validation and test sets.

Chapter 7 concludes the work, lists the future enhancements and describes the social impacts of the project. The appendices give the working environment and the source code, followed by the references.

1.6 SUMMARY

This chapter explained why sentiment classification of reviews is important and difficult, stated the objectives of the project, listed the outcomes and described the structure of the report. The next chapter surveys the literature that forms the basis of the proposed model.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

This section discusses the methodologies learnt from the previous works for the proposed system including machine learning approaches, sentiment classification and Transformer-based models. For every work the technique, its merits and its limitations are discussed, and a comparative summary is given at the end of the chapter.

2.2 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [6]

Before BERT, most NLP systems relied either on hand-engineered features or on word embeddings that gave every word a single fixed vector regardless of context. Devlin et al. argued that a language model should instead learn representations that depend on the full surrounding sentence, learned once on a very large unlabelled corpus and then reused for many downstream tasks rather than trained from scratch for each one.

In order to achieve this, BERT stacks many Transformer encoder layers, each built from multi-head self-attention followed by a feed-forward block, and pretrains the whole stack on two unsupervised objectives at once: masked-language-modelling, in which roughly 15% of input tokens are randomly replaced with a special mask token and the model must predict the original word from both left and right context, and next-sentence prediction, in which the model judges whether two given sentences originally followed one another in the source text.

After pretraining on large corpora such as BookCorpus and English Wikipedia, the same encoder is reused for a specific task by adding a small, task-specific output layer on top and fine-tuning the whole network end-to-end on labelled data, for classification, the output layer reads the final-layer representation of the special [CLS] token. Then the single pretrain-then-fine-tune recipe gave large, consistent improvements across many different NLP benchmarks compared with training a model from scratch on each task.

Advantages:

- Produces bidirectional, context-dependent word representations.

- Transfer learning from large-scale pretraining gives large accuracy gains on many downstream NLP tasks.
- Self-attention captures relationships between tokens regardless of their distance in the sequence.
- It has become the de facto standard backbone for nearly all subsequent encoder-based NLP research.

Disadvantages:

- The entire input is summarised into a single [CLS] token representation, which can lose fine-grained detail.
- Fine-tuning the full model is computationally expensive.
- Fine-tuning performance is sensitive to the random seed used.
- Also sensitive to the learning rate chosen.

2.3 DeBERTa: Decoding-enhanced BERT with Disentangled Attention [8]

He et al. observed that BERT adds position information to a token by summing a position embedding directly into the word embedding before the first layer, so content and position end up entangled in one vector for the rest of the network. There araise an arguement that a word's meaning and its position in the sentence are different kinds of information and should be allowed to interact with each other explicitly rather than being blended together from the very start.

DeBERTa therefore keeps two separate vectors for every token throughout the network: a content vector and a relative-position vector. The self-attention score between two tokens is computed as the sum of three separate terms, content-to-content, content-to-position and position-to-content, each using a different projection of these two vectors, rather than the single content-to-content term that standard self-attention computes. This disentangled formulation lets the model learn, for example, that two words three positions apart interact differently than two words next to each other.

Because relative position alone is not enough information for predicting a masked word near the start or end of a sentence, DeBERTa adds an enhanced mask decoder that additionally injects the token's absolute position just before the final output layer used during masked-language-model pretraining. The resulting pretrained encoders were evaluated on

the same GLUE-style benchmarks as BERT and consistently outperformed similarly sized BERT and RoBERTa models.

Advantages:

- Models word order and relative distance more accurately than standard BERT.
- Achieves stronger downstream accuracy than comparably sized BERT models.
- The enhanced mask decoder improves the quality of the pretraining signal.
- Disentangling content and position allows richer, more informative attention patterns.

Disadvantages:

- The additional disentangled attention terms increase memory requirements.
- Increases computation cost compared to standard self-attention.
- Still ultimately relies on a single pooled vector for classification.
- More complex to implement and tune than vanilla BERT.

2.4 DeBERTaV3: Improving DeBERTa using ELECTRA-style Pre-training with Gradient-Disentangled Embedding Sharing [9]

Masked-language-modelling only produces a training signal from the roughly 15% of tokens that were masked out, so most of the computation spent on the other 85% of tokens in every batch contributes nothing directly to the loss. DeBERTaV3 replaces this objective with replaced-token detection, following the ELECTRA, a small generator network proposes plausible replacement tokens for a subset of positions, and a larger discriminator network, which becomes the model that is actually kept and used later, and it must decide for every single token in the sequence whether it is original or was replaced by the generator.

But every position now contributes to the discriminator's loss, not just the masked ones, the discriminator receives far more learning signal per batch, which is what gives DeBERTaV3 its improved sample efficiency. A naive implementation would simply share the token-embedding matrix between the generator and discriminator, but the authors found this creates a 'tug-of-war' in which the two networks' gradients pull the shared embeddings in conflicting directions. Gradient-disentangled embedding sharing fixes this by letting the discriminator use the generator's embeddings as a fixed starting point while stopping the generator's gradients from flowing back through them during the discriminator's update.

The encoder architecture itself, a disentangled attention mechanism and the enhanced mask decoder, is otherwise the same as DeBERTa. The authors released a family of model

sizes, from extra-small to large, and evaluated them on the same GLUE and question-answering benchmarks used for DeBERTa and BERT, where DeBERTaV3 matched or beat much larger models while being considerably cheaper to fine-tune; the base-size variant is the encoder used as the backbone of the present project.

Advantages:

- Higher sample efficiency than the original DeBERTa.
- Better downstream accuracy for the same parameter budget.
- A compact, publicly available base model (deberta-v3-base) suits a modest hardware budget.
- Learns from every token in the input, not only masked positions.

Disadvantages:

- Fine-tuning still requires careful learning-rate selection.
- Mixed-precision training can be numerically delicate.
- Pretraining itself i.e generator plus discriminator is more complex than standard masked-language modelling.
- Remains a large model requiring non-trivial compute for fine-tuning.

2.5 Long Short-Term Memory [10]

Standard recurrent neural networks update a single hidden state at every time step by repeatedly multiplying it through the same weight matrix, and when this is unrolled over many time steps during back-propagation, the resulting gradients tend to either shrink toward zero or grow without bound. Hochreiter and Schmidhuber identified this vanishing and exploding gradient problem as the main reason ordinary recurrent networks of their time failed to learn dependencies that span more than a few steps.

The solution adds a separate memory cell that runs alongside the hidden state and is updated additively rather than through repeated matrix multiplication, which keeps gradients from vanishing as quickly. Three learned gates, an input gate, a forget gate and an output gate, each implemented as a sigmoid function of the current input and the previous hidden state, control respectively how much new information enters the memory cell, how much old information is retained, and how much of the memory cell is exposed as the hidden state used by the rest of the network.

Because the gates are learned rather than fixed, the same cell can choose to remember information for a few steps or for very long stretches depending on what the task requires. This made the LSTM the standard building block for sequence modelling tasks such as language modelling, speech recognition and machine translation for roughly two decades, and it remains the direct ancestor of later gated and ordered variants, including the ON-LSTM decoder used in this project.

Advantages:

- Solves the vanishing-gradient problem of plain recurrent networks.
- Can retain information over long sequences.
- Became the foundation for a large family of later gated recurrent architectures.
- Simple, well understood and widely implemented.

Disadvantages:

- Processes sequences strictly step by step, so it cannot be parallelised over time.
- Slower to train than Transformer-based alternatives on modern hardware.
- Has no explicit mechanism for separating long-term structure from local detail.
- Can still struggle with very long sequences in practice.

2.6 Ordered Neurons: Integrating Tree Structures into Recurrent Neural Networks ON-LSTM [11]

A standard LSTM treats every neuron in its hidden state identically: any neuron may be updated or left unchanged at any time step, with no built-in notion that some information such as the topic of a whole paragraph should persist much longer than other information such as the identity of the current word. Shen et al. argued that natural language has an implicit hierarchical, tree-like structure, and that a recurrent cell could benefit from a similar bias toward high-level information changing slowly and low-level information changing quickly.

The ON-LSTM encodes this bias by ordering the neurons of the memory cell from 0 to the cell size and introducing two master gates, a master forget gate and a master input gate, each computed with a cumulative softmax (cumax) function so that the gate value is monotonically non-decreasing for the forget gate or non-increasing for the input gate along the neuron order. A high-ranked neuron is therefore forgotten less easily and updated less often than a low-ranked neuron, which means high-ranked neurons naturally come to

represent long-lived, high-level structure while low-ranked neurons represent fast-changing, local detail.

The overlap between where the master forget gate is already high and where the master input gate is still high defines a region in which the standard per-neuron gates (as in a normal LSTM) are allowed to operate; outside that region the master gates alone decide whether a neuron is kept or overwritten. The authors evaluated the resulting cell on language modelling, where it reduced perplexity compared with a standard LSTM of the same size, on unsupervised constituency parsing, where the induced neuron ordering was shown to recover tree structures resembling human-annotated parses, and on a logical inference task that specifically requires tracking nested structure.

Advantages:

- The ordered, hierarchical memory gives a useful inductive bias for naturally nested language.
- It is used as a helpful language modelling.
- The model is useful in unsupervised constituency parsing.
- It also helps in logical-inference tasks.

Disadvantages:

- Still must be unrolled token by token, so it is not parallelisable.
- Slower to run than an equivalent Transformer layer.
- Has more gate parameters than a standard LSTM cell.
- Adds implementation complexity over a plain LSTM or GRU.

2.7 Adversarial Training Methods for Semi-supervised Text Classification [13]

Adversarial training was first proposed for image classifiers, where a small, carefully chosen perturbation added to the pixels of an input image, invisible to a human but aligned with the direction that most increases the model's loss, could cause the model to misclassify the image; training on a mixture of clean and such adversarial images was shown to make image classifiers both more robust and more accurate. Miyato, Dai and Goodfellow set out to bring the same idea to text classification, where the direct analogue does not exist because text is made of discrete tokens that cannot be nudged by a small continuous amount.

The solution is to apply the perturbation not to the discrete input tokens but to the continuous word-embedding vectors, which can be perturbed just like image pixels. During

training, a normal forward and backward pass is first run to obtain the gradient of the loss with respect to the embeddings; the embeddings are then shifted a small distance in the direction of this gradient, normalised to a fixed size, and a second forward and backward pass is run on the perturbed embeddings, with both passes' gradients contributing to the parameter update. The paper also introduces a virtual adversarial variant that applies the same idea to unlabelled data by perturbing in the direction that most changes the model's predicted output distribution rather than the labelled loss, allowing the technique to be used for semi-supervised learning.

The method was evaluated on standard text classification benchmarks, including IMDB movie reviews, where models trained with the adversarial and virtual-adversarial objectives achieved lower error rates than the same architecture trained without perturbation, and also produced word embeddings that were qualitatively more semantically coherent. This embedding-perturbation recipe, one additional forward and backward pass using the normalised loss gradient, became the standard template for later adversarial training methods in NLP, including the FGM step used in the present project.

Advantages:

- A cheap regulariser, needing only one extra forward and backward pass per step.
- Directly applicable to embeddings, sidestepping the problem of perturbing discrete text.
- Improves robustness and generalisation of neural text classifiers.
- Became the template later adopted as the FGM step used in projects such as this one.

Disadvantages:

- Roughly doubles the per-step training time.
- An unsuitable perturbation size can hurt rather than help accuracy.
- The perturbed embeddings do not correspond to any real word.
- Requires tuning of the perturbation-strength hyper-parameter.

2.8 Algorithms for Hyper-Parameter Optimization [14]

Hyper-parameters for machine-learning models were typically chosen by manual trial and error, grid search over a fixed set of values, or plain random search. Bergstra et al. pointed out that grid search scales very poorly as the number of hyper-parameters grows, that manual search does not scale across many experiments, and that neither approach uses

information from earlier trials to decide what to try next, so both waste a large fraction of the evaluation budget on clearly unpromising regions of the search space.

The proposed alternative, the Tree-structured Parzen Estimator (TPE), is a sequential model-based optimisation method: instead of directly modelling how the validation score depends on the hyper-parameters, it splits the trials observed so far into two groups using a quantile threshold on the observed score, a 'good' group and a 'bad' group, and fits a separate density over the hyper-parameter values in each group. The next configuration to try is chosen to maximise the ratio of the good-group density to the bad-group density, which favours regions that tend to produce good results while still allowing exploration of less-visited areas.

Because TPE builds a tree-structured representation of the search space, it naturally supports conditional hyper-parameters, where one parameter only exists if another parameter takes a particular value, something grid search handles awkwardly. The authors demonstrated TPE on tuning deep belief networks and related architectures, where it found better configurations than manual and random search within the same evaluation budget, and the algorithm later became the default sampling strategy inside hyper-parameter optimisation frameworks such as Optuna.

Advantages:

- More sample-efficient than grid search or random search.
- Builds a probabilistic model that improves as more trials complete.
- Works well with conditional and structured search spaces.
- Forms the theoretical basis for the TPE sampler used in modern tools such as Optuna.

Disadvantages:

- Needs a reasonable number of initial trials before its model becomes informative.
- More complex to implement than simple random search.
- Can get stuck favouring regions found early if too little exploration occurs.
- Provides no mechanism on its own for early-stopping poor trials.

2.9 Hyperband: A Novel Bandit-Based Approach to Hyper-Parameter Optimization [16]

Model-based methods such as TPE decide which configuration to try next but still train every configuration for the same, usually generously chosen, number of epochs before

judging it, which wastes compute on configurations that are clearly bad from a much earlier point. Li et al. instead treated hyper-parameter search as a pure resource-allocation problem: given a fixed total compute budget, how should it be split between trying more configurations for a shorter time each versus trying fewer configurations for longer.

Their algorithm, Hyperband, is built on top of a subroutine called successive halving: start a set of configurations training with a small initial budget, measure their validation performance, discard the worst fraction (for example the bottom half), double the budget given to the survivors, and repeat until only one configuration remains or the maximum budget is reached. Because the right trade-off between the number of starting configurations and the initial budget per configuration is not known in advance, Hyperband runs several brackets of successive halving in parallel, each with a different starting trade-off, so that at least one bracket is close to the right choice for the problem at hand.

This makes Hyperband conceptually simple, easy to parallelise, and free of any assumption that validation performance changes smoothly with the hyper-parameters, unlike Bayesian methods such as TPE. The authors showed it could find configurations of comparable quality to Bayesian optimisation methods using a fraction of the compute on tasks such as convolutional network and kernel-method tuning, and its successive-halving, early-stopping logic is the direct conceptual ancestor of the pruning strategies used in later tools such as Optuna.

Advantages:

- Allocates computing budget efficiently by discarding poor configurations early.
- Requires no assumptions about the smoothness of the objective function.
- Simple to implement relative to full Bayesian-optimisation methods.
- Provides the conceptual basis for the pruning strategies used in tools such as Optuna.

Disadvantages:

- Early performance is not always a reliable predictor of final performance, so good configurations can be discarded too soon.
- Is purely resource-based and does not use information about which hyper-parameter regions tend to work well.
- Requires choosing bracket and budget schedules, which themselves need some tuning.
- Can waste resources on brackets with very aggressive early stopping when the task is noisy.

2.10 Optuna: A Next-generation Hyperparameter Optimization Framework [15]

Earlier hyper-parameter optimisation libraries typically required the search space to be declared up front, as a fixed, static configuration object, before the objective function could be written, which makes it awkward to express search spaces where some hyper-parameters only make sense conditionally on others, such as searching over the number of layers in a network and then, separately, the size of each layer that actually exists. Akiba et al. designed Optuna around a 'define-by-run' philosophy instead, in which the search space is defined by ordinary Python code written directly inside the objective function.

On each call, the objective function asks a trial object to 'suggest' a value for each hyper-parameter as it is needed, and the underlying sampler, by default a TPE sampler in the spirit of Bergstra et al.'s work, uses the history of previous trials to propose each value. At the same time, the objective function can periodically report its intermediate performance back to Optuna, which feeds a pruning algorithm such as the median pruner, conceptually descended from Hyperband's successive halving, that can terminate a clearly unpromising trial early rather than letting it run to completion.

Optuna also stores every trial's parameters and results in a persistent backend such as an SQLite database, so a long search can be paused and resumed, or run in parallel across multiple processes or machines that all write to the same storage. The authors demonstrated the framework on a range of real-world machine-learning tuning problems, and it is this combination of a flexible, code-based search-space definition with an efficient sampler and pruner that the present project uses to search the ten hyper-parameters of the proposed model.

Advantages:

- The define-by-run API allows flexible, dynamic search spaces.
- Combines an efficient sampler (TPE) with early pruning (Hyperband-like) in a single framework.
- Persistent storage (e.g. SQLite) allows an interrupted search to be resumed.
- This is the actual hyper-parameter search tool used in the project.

Disadvantages:

- With very few trials the sampler has little information to act on.
- Result quality depends heavily on the computing budget available.
- Noisy intermediate scores can cause genuinely good trials to be pruned.

- Needs careful configuration of the pruner's warm-up and start-up parameters to avoid premature pruning.

2.11 TRALSem: A Robust Model for Textual Sentiment Analysis [25]

Yang et al. set out to build a sentiment classifier that is both accurate and robust to small, incidental changes in its input and training conditions, two properties that ordinary fine-tuned Transformer classifiers do not automatically have. Their starting point is a standard 12-layer BERT-style Transformer encoder, fine-tuned on sentiment data in the usual way, but adding two mechanisms on top of this base, aimed specifically at reproducibility and at robustness to small input perturbations.

For reproducibility, the model uses a linear-congruential random-seed strategy that generates a deterministic sequence of seeds for data shuffling and weight initialisation, so that repeated runs of the same configuration behave consistently rather than varying unpredictably with the system's default random state. For robustness, training incorporates adversarial perturbations of the word embeddings, and the authors evaluate four different strategies for generating these perturbations, the single-step FGM method, the multi-step PGD method, the larger-step FreeLB method, and the first-layer-only YOPO method, reporting results for whichever strategy performed best for a given dataset.

Rather than the usual softmax output layer trained with cross-entropy loss, TRALSem adds a single linear output unit trained with a root-mean-square-error regression loss against a numeric sentiment target, which the authors argue gives a smoother training signal than a hard classification boundary. The resulting model was evaluated on IMDB as well as Chinese and other English sentiment datasets, reporting a macro-F1, micro-F1, precision and recall of 0.9523 on IMDB, and the authors compare it against several other pretrained and lexicon-based baselines.

Advantages:

- Combines a pretrained Transformer encoder with adversarial training in one robust pipeline.
- A reproducible seeding strategy makes results easier to replicate.
- Reports strong results (macro-F1 of 0.9523 on IMDB) and improves over several baselines on other datasets.

- The closest existing system to the proposed model, providing a concrete point of reference.

Disadvantages:

- Uses fixed hyper-parameters rather than an automatic search.
- Has no sequential decoder to model how sentiment develops through a review.
- Reports results from a single configuration rather than a validated, tuned one.

2.12 COMPARISON TABLE OF SURVEYED WORKS

Table 2.1 Comparative summary of the surveyed works

Author Name	Methodology Used	Limitations
Devlin et al. [6]	BERT — masked-language modelling with bidirectional context, fine-tuned per task.	Expensive to fine-tune; sensitive to seed and learning rate.
He et al. [8]	DeBERTa — disentangled content/position attention with an enhanced mask decoder.	Higher memory/compute cost; still relies on one pooled vector.
He et al. [9]	DeBERTaV3 — ELECTRA-style replaced-token detection with gradient-disentangled embeddings.	Pretraining is more complex; needs non-trivial compute.
Hochreiter & Schmidhuber [10]	LSTM — gated memory cell to avoid vanishing/exploding gradients.	Sequential, cannot be parallelised; no long/short-term separation.
Shen et al. [11]	ON-LSTM — cumulative-softmax master gates give an ordered, hierarchical memory.	Not parallelisable; more gate parameters than a plain LSTM.
Miyato, Dai & Goodfellow [13]	FGM — perturbs word embeddings along the loss gradient as a regulariser.	Doubles per-step training time; wrong strength can hurt accuracy.
Bergstra et al. [14]	TPE — models good/bad hyper-parameter regions as separate densities.	Needs several trials before it is informative; no early stopping.
Li et al. [16]	Hyperband — successive halving discards poor configurations early.	Early performance not always predictive; ignores which regions work well.
Akiba et al. [15]	Optuna — define-by-run framework combining TPE sampling with pruning.	Needs enough trials; noisy scores can prune good ones.
Yang et al. [25]	TRALSem — BERT encoder with reproducible seeding and adversarial training.	Fixed hyper-parameters; no sequential decoder; single configuration.

2.13 SUMMARY

This chapter had given the description about the literature survey for various existing techniques involved in Transformers, Adversarial Training, Optimization, Hyper-parameter tuning. Next chapter deals with the system study for the proposed system.

CHAPTER 3

SYSTEM STUDY

3.1 OVERVIEW

This chapter provides a concise introduction of the terms and technologies used in the proposed system, including classical machine learning approaches, Recurrent and convolutional approaches, their limitations, with a comparison table also this modules discuss how the proposed system works.

3.2 EXISTING METHOD

3.2.1 Classical machine learning approaches

The first generation of sentiment classifiers represented each review by the frequency or presence of words and short word sequences and trained a Naive Bayes, logistic regression or support vector machine classifier [1]. These systems train quickly and are easy to interpret, but they treat words independently and require feature engineering, and cannot recognise that “not good” means the opposite of “good”.

3.2.2 Recurrent and convolutional neural approaches

Neural networks replaced sparse features by dense word vectors. Convolutional networks detect local phrases [5], while LSTM networks read the sentence left to right and maintain memory that carries information across many words [10]. Attention mechanisms are used to allow such models to concentrate on the most informative words [17]. Their weakness is that the word embeddings are not contextual and are usually trained on limited data.

3.2.3 Transformer fine-tuning with single-vector pooling

At present the strongest baselines fine-tune a pretrained Transformer encoder [6, 8]. The review is tokenised, passed through the encoder, and the vector of the first token or the mean of all token vectors is given to a linear layer with softmax, as shown in Figure 3.1. This approach achieves high accuracy with little task-specific design.

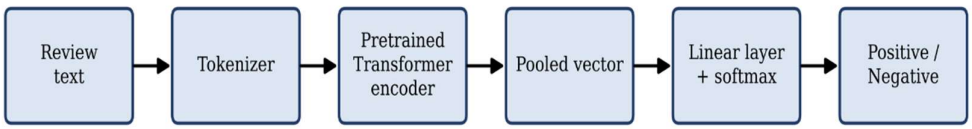


Figure 3.1 Block diagram of the existing Transformer fine-tuning approach

3.2.4 Limitations of the existing methods

- Classical and convolutional models can't represent long-range dependencies such as negation separated from its target.
- Single-vector pooling compresses the entire review at once and does not model how sentiment evolves from clause to clause.
- Standard fine-tuning has no explicit protection against small perturbations of the embeddings, which may reduce robustness.
- Performance depends on many hyper-parameters, and some settings (for example a high learning rate) make training collapse [3]. Manual tuning is slow and wastes computing resources (SDG 12).

3.3 PROPOSED METHOD

The proposed model extends the existing fine-tuning pipeline with an ordered-memory decoder, a learned pooling mechanism, adversarial regularisation and automatic hyper-parameter selection. Figure 3.2 shows the overall workflow, and the following subsections describe each stage.

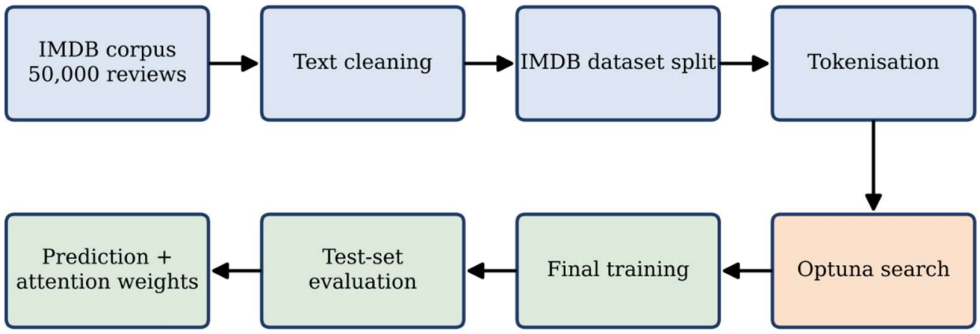


Figure 3.2 Workflow of the proposed model

3.3.1 Text cleaning and encoding

Raw IMDB reviews contain HTML line breaks, entities and sometimes web addresses. A cleaning function decodes entities, removes tags and URLs, normalises the Unicode form and collapses whitespace. The cleaned corpus is divided into stratified training, validation and test parts, tokenised with the DeBERTa-v3 tokeniser, truncated to 128 tokens and padded dynamically within every mini-batch.

3.3.2 DeBERTa contextual encoding

The pretrained DeBERTa-v3-base encoder converts every token into a 768-dimensional contextual vector. The encoder is fine-tuned together with the new layers, using a smaller learning rate than the new layers so that the pretrained knowledge is preserved.

3.3.3 ON-LSTM decoding

The encoder output is regularised with dropout and processed by an ON-LSTM decoder. The decoder reads the tokens in order and maintains a memory whose neurons are ordered by the time scale of the information they hold. Slowly changing neurons can retain the general opinion of the review, whereas fast neurons follow the current phrase.

3.3.4 Attention pooling and classification

Instead of using the last or the first state, a learned additive attention layer assigns a weight to each token and builds a weighted sum of the decoder outputs. The weights are normalised only over real tokens. A small fully connected block with GELU activation and dropout maps the review vector to two logits, and softmax produces the probabilities of the negative and positive classes.

3.3.5 FGM adversarial training

In every training step the loss is first computed on the clean input. The word embeddings are then moved a small distance in the direction of the normalised gradient, the loss is computed again on the perturbed input and its gradients are added to the first ones, and finally the embeddings are restored. The model therefore learns to classify correctly both the original and the perturbed representations.

3.3.6 Hyper-parameter optimisation with Optuna

An Optuna study maximises the validation F1-score of short training runs. A TPE sampler proposes the learning rates, weight decay, warm-up ratio, dropout, ON-LSTM size and adversarial strength, and a median pruner stops trials that fall behind. The best configuration is stored in a JSON file and used for the final training run on the whole training split.

3.4 COMPARISON OF EXISTING AND PROPOSED METHODS

Table 3.1 Comparison between the existing and the proposed method

Aspect	Existing (single-vector pooling)	Proposed (this project)
Encoder	Pretrained Transformer	DeBERTa-v3-base, fine-tuned

Aspect	Existing (single-vector pooling)	Proposed (this project)
Sequence modelling	None after the encoder	ON-LSTM decoder with ordered memory
Pooling	[CLS] token or mean	Learned attention over real tokens
Robustness	No embedding perturbation	FGM adversarial training
Hyper-parameters	Selected manually	Optuna (TPE + median pruner)
Interpretability	Limited	Token attention weights available

3.5 SUMMARY

The chapter discussed about the various terms and technologies related to existing system, and the modules that will be used in the proposed system. The comparison table with the existing and proposed work is also provided, the next chapter discusses about the proposed system.

CHAPTER 4

SYSTEM DESIGN

4.1 OVERVIEW

This chapter gives the detailed design of the proposed system. It describes the overall architecture, the preprocessing steps, and the mathematical formulation of the encoder, the ON-LSTM decoder, the attention-pooling layer, the classifier, the adversarial training module and the hyper-parameter optimisation module.

4.2 SYSTEM ARCHITECTURE

Figure 4.1 shows the architecture of the proposed system. An input review is tokenised by the DeBERTa tokeniser and passed to the DeBERTa encoder, which consists of a stack of Transformer blocks. The adversarial training block is attached to the encoder and perturbs its embedding layer during training. The token representations flow into the ON-LSTM decoder, which is made of K memory layers, then into the attention-pooling layer, the fully connected layer and the softmax layer, which outputs the probabilities of the positive and negative classes. The Optuna block above the encoder represents the hyper-parameter optimiser, whose objective function, sampler (TPE) and pruner control the settings used by the training run.

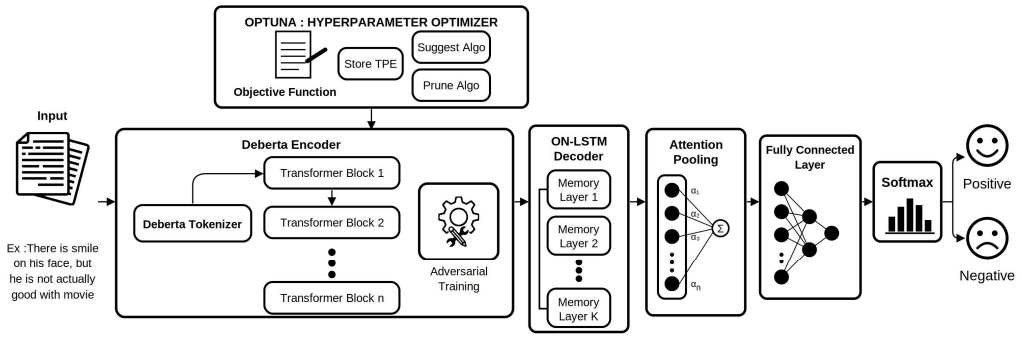


Figure 4.1 System architecture of the proposed model

4.3 DATASET AND DATA PREPROCESSING

4.3.1 Dataset

The IMDB Large Movie Review dataset [2, 4] contains 50,000 reviews labelled as positive or negative, with exactly 25,000 reviews of each class. The two halves supplied with the dataset are merged and re-split in a stratified manner with a fixed seed

of 42, producing 35,000 training, 5,000 validation and 10,000 test reviews (Table 4.1). Stratification keeps the class proportions identical in all parts.

Table 4.1 Stratified split of the IMDB corpus

Split	Share	Reviews	Positive	Negative
Training	70 %	35,000	17,500	17,500
Validation	10 %	5,000	2,500	2,500
Test	20 %	10,000	5,000	5,000
Total	100 %	50,000	25,000	25,000

4.3.2 Text cleaning

This process describes Data pre-processing and Tokenization : (i) HTML entities are decoded; (ii) HTML tags such as the line-break marker are replaced by a space; (iii) web addresses are replaced by a space; (iv) the text is converted to the Unicode NFKC normal form; and (v) consecutive whitespace characters are collapsed and the ends are trimmed. These operations remove noise that carries no sentiment while keeping the wording of the review unchanged.

4.3.3 Tokenisation and batching

The cleaned text is tokenised with the DeBERTa-v3 tokeniser, which uses a SentencePiece vocabulary and adds the [CLS] and [SEP] markers. Sequences are truncated to a maximum length of 128 tokens. To save computation, padding is not applied at tokenisation time; instead a collate function pads each mini-batch to the length of its longest sequence and creates an attention mask in which real tokens are 1 and padding is 0 (Figure 4.2).

4.4 DEBERTA ENCODER

Let $X = [x_1, x_2, \ldots, x_n]$ denote the token sequence of a review. The pretrained encoder transforms it into a matrix of contextual vectors given by Equation (4.1),

$$H = \mathrm{DeBERTa}(X), \quad H \in \mathbb{R}^{T \times d} \tag{4.1}$$

where T is the sequence length and d = 768 is the hidden size of the base model. All encoder weights are fine-tuned. Dropout is applied to H before it enters the decoder so that the new layers do not depend on individual encoder features.

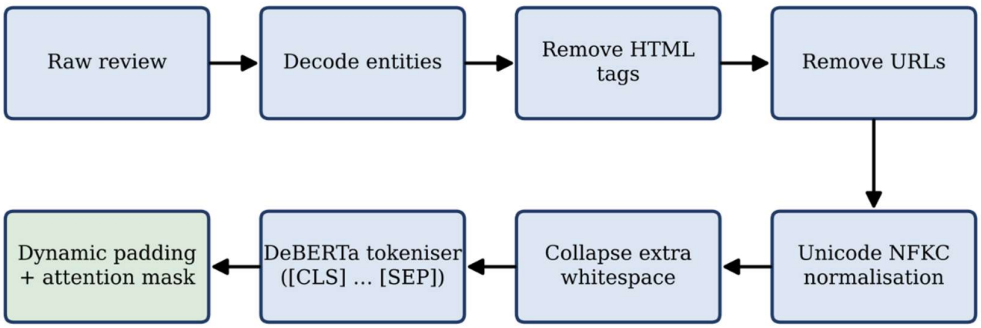


Figure 4.2 Data preprocessing and batching pipeline

4.5 ON-LSTM DECODER

The decoder reads H one position at a time. Its distinguishing function is the cumulative softmax, cumax, which converts a score vector into a non-decreasing sequence between 0 and 1 (Equation 4.2):

$$cumax(u)_k = \sum_{j=1}^{k} softmax(u)_j \tag{4.2}$$

The hidden units are divided into chunks, and two master gates are computed per chunk from the current input x_t and the previous hidden state h_{t-1}. The master forget gate increases along the neuron order, whereas the master input gate decreases (Equations 4.3 and 4.4):

$$\tilde{f}_t = cumax(W_{\tilde{f}}\, x_t + U_{\tilde{f}}\, h_t - 1 + b_{\tilde{f}}) \tag{4.3}$$

$$\tilde{\imath}_t = 1 - cumax(W_{\tilde{\imath}}\, x_t + U_{\tilde{\imath}}\, h_t - 1 + b_{\tilde{\imath}}) \tag{4.4}$$

Their element-wise product defines the region where both gates are active (Equation 4.5), and it is used to merge the master gates with the standard forget gate f_t and input gate i_t (Equations 4.6 and 4.7):

$$\omega_t = \tilde{f}_t \circ \tilde{\imath}_t \tag{4.5}$$

$$\hat{f}_t = f_t \circ \omega_t + (\tilde{f}_t - \omega_t) \tag{4.6}$$

$$\hat{\imath}_t = i_t \circ \omega_t + (\tilde{\imath}_t - \omega_t) \tag{4.7}$$

The standard gates and the candidate content are computed for every neuron, where σ is the logistic sigmoid and $\circ$ is the element-wise product or Haramard product (Equation 4.8):

$$
\begin{aligned}
f_t &= \sigma(W_f\, x_t + U_f\, h_t - 1 + b_f) \\
i_t &= \sigma(W_i\, x_t + U_i\, h_t - 1 + b_i) \\
o_t &= \sigma(W_o\, x_t + U_o\, h_t - 1 + b_o) \\
\hat{c}_t &= tanh(W_c\, x_t + U_c\, h_t - 1 + b_c)
\end{aligned}
\tag{4.8}
$$

The memory cell and hidden state are then updated by Equations (4.9) and (4.10). Figure 4.3 illustrates the complete cell.

$$c_t = \hat{f}_t \circ c_t - 1 + \hat{\imath}_t \circ \hat{c}_t \tag{4.9}$$

$$h_t = o_t \circ tanh(c_t) \tag{4.10}$$

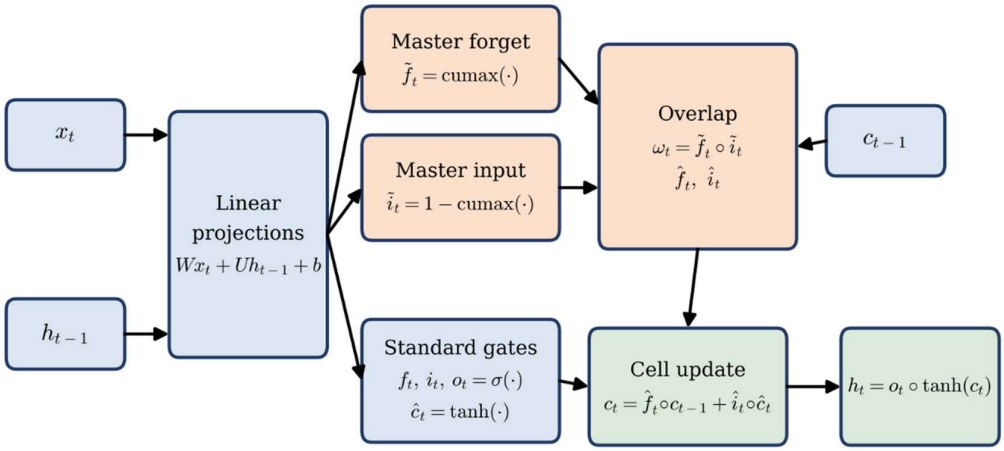


Figure 4.3 Structure of the ON-LSTM cell

Reviews in a mini-batch have different lengths. A padding mask $m_t \in \{0, 1\}$ therefore stops padded positions from changing the memory: for a padded position the previous state is carried forward unchanged and the output is set to zero (Equation 4.11), where $\tilde{h}_t$ and $\tilde{c}_t$ are the newly computed states of Equations (4.9) and (4.10).

$$
\begin{aligned}
h_t &= m_t\, \tilde{h}_t + (1 - m_t) h_{t-1} \\
c_t &= m_t\, \tilde{c}_t + (1 - m_t) c_{t-1}
\end{aligned}
\tag{4.11}
$$

When more than one memory layer is used, the output sequence of one layer becomes the input of the next layer, with dropout between the layers. The decoder output is the matrix Z = $(z_1, \ldots, z_T)$.In the final model one memory layer with 128 hidden units in chunks of eight neurons is used.

4.6 ATTENTION POOLING AND CLASSIFIER

Rather than depending on the last hidden state, which may belong to padding or to an uninformative token, the model learns the importance of every token. A relevance score is computed by a small feed-forward scorer (Equation 4.12) and normalised by a softmax that is restricted to non-padded positions so that padding gets zero weight (Equation 4.13). The review vector is the weighted sum of the decoder outputs (Equation 4.14), as shown in Figure 4.4.

$$e_t = v_\top \tanh(W_a z_t + b_a) \tag{4.12}$$

$$\alpha_t = softmax(e_1, \ldots, e_n) \tag{4.13}$$

$$s = \sum_{t=1}^{T} \alpha_t z_t \tag{4.14}$$

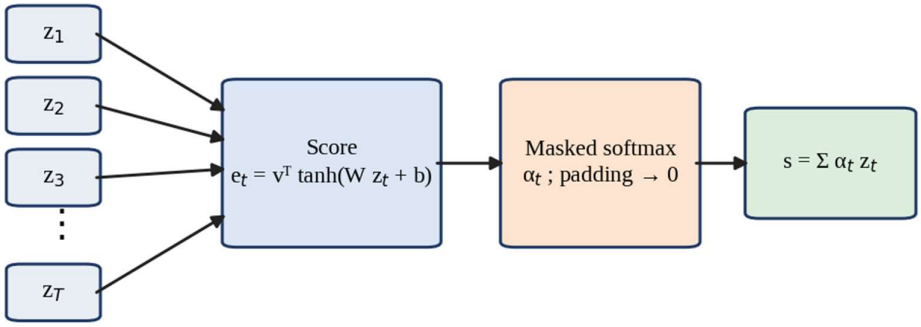


Figure 4.4 Attention-pooling mechanism

The review vector s passes through a fully connected block consisting of dropout, a hidden layer of half the decoder width with GELU activation, a second dropout layer and an output layer with two units. Softmax gives the class probabilities (Equation 4.15), and the parameters are trained by minimising the cross-entropy loss over a mini-batch of N reviews (Equation 4.16):

$$p = softmax(W_2\, GELU(W_1 s + b_1) + b_2) \tag{4.15}$$

$$L(\theta) = -\left(\frac{1}{N}\right) \sum_{i=1}^{N} \log p_i, y_i \tag{4.16}$$

Here θ stands for all trainable parameters and y_i is the true class of review i. The attention weights α_t also allow the contribution of each token to be inspected.

4.7 ADVERSARIAL TRAINING MODULE

To make the classifier less sensitive to small changes of its input representation, an adversarial perturbation is added to the word-embedding matrix E. After a normal forward and backward pass has produced the gradient g of the loss with respect to E, the embeddings are shifted in the direction that increases the loss, the size of the shift being controlled by ε (Equation 4.17):

$$r_{adv} = \frac{\varepsilon \cdot g_E}{\|g_E\|_2} \ , \ g = \nabla E \ L(\theta; E) \tag{4.17}$$

A second forward and backward pass is carried out with the perturbed embeddings E + r_{adv}. Its gradients are accumulated on top of those of the clean pass, so that the optimiser effectively minimises the sum of the clean and adversarial losses (Equation 4.18). The original embeddings are restored before the parameter update. Figure 4.5 summarises one training step.

$$L_{total} = L(\theta; E) + L(\theta; E + r_{adv}) \tag{4.18}$$

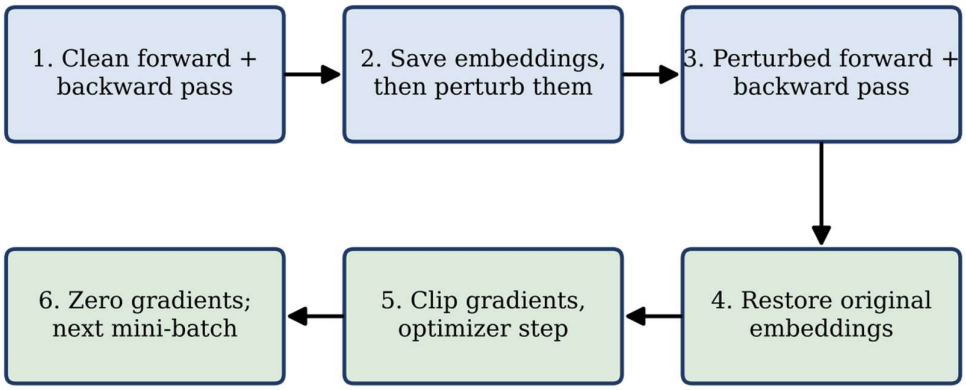


Figure 4.5 One training step with FGM adversarial training

If the gradient norm is zero or not finite, which can happen under half-precision arithmetic, the perturbation is skipped for that step. Setting ε = 0 disables adversarial training.

4.8 HYPER-PARAMETER OPTIMIZATION MODULE

The hyper-parameters are chosen by maximising the validation F1-score of short trials (Equation 4.21), where λ is a configuration drawn from the search space Λ:

$$\lambda^* = argmax \ \lambda \in \Lambda \ F1_{val}(\lambda) \tag{4.21}$$

Figure 4.6 shows the procedure. A study is created with a TPE sampler and a median pruner and stored in an SQLite database, so that an interrupted search can be resumed. For every trial the sampler suggests a configuration, the model is trained for two epochs, and the validation F1-score is reported after each epoch. If the score is below the median of earlier trials at the same epoch, the trial is pruned; otherwise it is completed and its score is stored. After the last trial the best parameters are written to a JSON file and a table of all trials is exported. Because a pruned trial stops early, the computing time and therefore the energy used for the search are reduced (SDG 12).

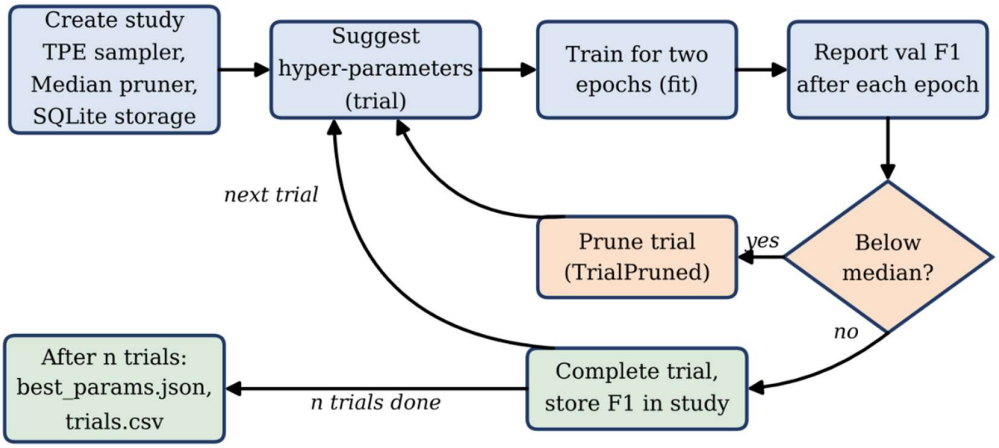


Figure 4.6 Optuna hyper-parameter optimisation workflow

The final search covered ten hyper-parameters, namely the eight listed in Table 5.3 together with the number of ON-LSTM layers {1, 2} and the batch size {16, 32}. Each trial trained for up to six epochs on a 10 % subset of the training and validation splits (3,500 and 500 reviews) with a maximum length of 256 tokens, and the score of a trial was its best validation F1-score over its epochs. Six trials were run with the same TPE sampler (seed 42) and median pruner (three start-up trials, one warm-up epoch). The best configuration was then used to train the final model on the complete training split.

4.9 SUMMARY

The chapter explained about the architecture design, modules and overall process of the work with mathematical representation and flow diagram for each modules. The next chapter discusses about the procedures and algorithms used in the work.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 OVERVIEW

This chapter explains how the design of Chapter 4 was implemented and the algorithms with explanation and corresponding outputs are discussed here. Three sample IMDB reviews are used throughout to illustrate the output produced by each module:

Review A — "There is a smile on his face, but he is not actually good with movie"; **Review B** — "The acting was brilliant and the story kept me hooked till the end. Loved it"; **Review C** — "What a waste of time & money. The plot was boring, see [URL]". Vector-valued outputs are shown truncated to their first few components (the full dimensionality is noted below each table) and are laid out as a 1×n matrix, i.e. a single row of a 2-dimensional array, so the shape of each representation is explicit.

5.2 DATA PROCESSING AND TOKENIZATION

Each review in the IMDB dataset is first cleaned by decoding any HTML entities and stripping HTML tags and URLs, then the text is normalised to Unicode NFKC form with repeated whitespace collapsed. Once cleaned, the full dataset is split into training (70%), validation (10%) and test (20%) sets. Each review in every split is then passed through the DeBERTa tokenizer, which breaks it into sub-word tokens using SentencePiece, after which a [CLS] token is added at the start and a [SEP] token at the end, and the sequence is truncated to a fixed maximum length. Every token is mapped to its vocabulary index to form the input sequence. Finally, within each mini-batch, shorter sequences are padded with [PAD] tokens up to the length of the longest sequence in that batch, and a binary attention mask is generated that marks real tokens as 1 and padded positions as 0, so the encoded train, validation and test sets are ready for the model.

Algorithm 1 Data Pre-processing and Tokenization

Input: IMDB movie-review dataset D, DeBERTa tokenizer T, maximum length L

Output: Encoded train, validation and test sets of (input_ids, _{attention}_mask, label)

for each review r in D

r = HTML-unescape(r) // Remove HTML tags (e.g.
) and URLs from r

r = Unicode NFKC-normalise(r) // Removes repeated whitespace , strip r

Split D into D_{train} (70 %), D_{test} (20 %) and D_{val} (10 %)

for each split S ∈ {D_{train}, D_{val}, D_{test}}

for each review r in S

tokens = T(r) // sub-word (SentencePiece) tokenization

Add [CLS] at the start and [SEP] at the end

Truncate tokens to L tokens

X = [$x_1, x_2, \ldots, x_n$] // map each token to its vocabulary index

for each mini-batch

Pad every X with [PAD] to the longest sequence in the batch

M_t = 1 for real tokens

M_t = 0 for padding //Attention Mask

Return encoded D_{train}, D_{val}, D_{test}

Sample Output — Review A: "There is a smile on his face, but he is not actually good with movie"

Stage	Output
Cleaned text	There is a smile on his face, but he is not actually good with movie
Tokens	[CLS] ▁There ▁is ▁a ▁smile ▁on ▁his ▁face , ▁but ▁he ▁is ▁not ▁actually ▁good ▁with ▁movie [SEP]
Input IDs (1×18 matrix)	[1, 443, 269, 266, 3292, 277, 315, 812, 261, 304, 313, 269, 298, 675, 397, 275, 1421, 2]
Attention Mask (1×18 matrix)	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Sample Output — Review B: "The acting was brilliant and the story kept me hooked till the end. Loved it"

Stage	Output
Cleaned text	The acting was brilliant and the story kept me hooked till the end. Loved it
Tokens	[CLS] ▁The ▁acting ▁was ▁brilliant ▁and ▁the ▁story ▁kept ▁me ▁hooked ▁till ▁the ▁end . ▁Loved ▁it ! [SEP]
Input IDs (1×19 matrix)	[1, 279, 3754, 284, 4269, 263, 262, 697, 1756, 351, 10826, 3925, 262, 513, 260, 17249, 278, 300, 2]
Attention Mask (1×19 matrix)	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Sample Output — Review C: "What a waste of time & money. The plot was boring, see [URL]"

Stage	Output
Cleaned text	What a waste of time & money. The plot was boring, see
Tokens	[CLS] ▁What ▁a ▁waste ▁of ▁time ▁& ▁money . ▁The ▁plot ▁was ▁boring , ▁see [SEP]
Input IDs (1×16 matrix)	[1, 458, 266, 2150, 265, 326, 429, 580, 260, 279, 4278, 284, 6715, 261, 398, 2]
Attention Mask (1×17 matrix)	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

5.3 DEBERTA ENCODER

With the pretrained DeBERTa-v3-base weights, each token is first converted into a content vector through word embedding, with positional information handled separately through relative position embeddings rather than being added directly to the word vector. The sequence then passes through twelve Transformer blocks; within each block, every pair of tokens has its attention score computed as the sum of three components — content-to-content, content-to-position and position-to-content — which together form the disentangled attention mechanism. These scores are scaled, and any padded position is masked out by setting its score to negative infinity before the softmax is applied, so padding tokens receive no attention. The resulting attention-weighted values are combined with the input through a residual connection and layer

normalisation, followed by a feed-forward sub-layer with its own residual connection and normalisation. After all twelve blocks, the output of the final layer for every token forms the contextual representation matrix H, of shape (sequence length × 768), that is passed on to the next module.

Sample Output — contextual representation H (truncated to the first 5 of 768 dimensions)

Review	Token	H — 1×5 row of the (n×768) matrix H
A	H[CLS]	[0.1368, 0.2143, 0.0260, -0.0358, 0.0872]
A	H[There]	[-0.5937, -0.0801, -0.0892, -0.9752, 0.3787]
B	H[CLS]	[0.3911, 0.2688, 0.0475, -0.0311, 0.0742]
B	H[The]	[1.0908, 0.0500, 0.1680, -0.8739, 0.0586]
C	H[CLS]	[0.1559, 0.1284, 0.1515, -0.0597, 0.1814]
C	H[What]	[0.4089, 0.1812, -0.0377, -0.1305, 0.0906]

Algorithm 2 :DeBERTa Encoder

Input: Token indices X = $[x_1, \dots, x_n]$, attention mask M
Output: Contextual token representations H = $[h_1, \dots, h_n]$, $h_t \in R^{768}$

//Load pre-trained DeBERTa-v3-base weights (12 Transformer blocks, hidden size 768)

for each token x_t

c_t^0 = WordEmbedding(x_t) // content vector; position is handled by relative embeddings P

for each Transformer block l = 1 to 12

for each token pair (i, j)

$A_{ij} = Q_i^c * K_j^c + Q_i^c * K_{i-j}^r + K_j^c * Q_{j-i}^r$

// content-to-content + content-to-position + position-to-content (disentangled attention)

$A_{ij} = \frac{A_{ij}}{\sqrt{3d}}$ // d is the size of query, key vectors

if $M_j = 0$ then $A_{ij} = -\infty$ // mask padding

α = softmax(A)

c^l = LayerNorm(c^{l-1} + α V)

c^l = LayerNorm(c^l + FeedForward(c^l))

$h_t = c_t^{12}$ for every token t

Return H

5.4 ON-LSTM DECODER

The encoder's output is divided conceptually into a number of neuron chunks and used as the input to the first of several stacked memory layers, each of which processes the sequence one time step at a time starting from zero-initialised hidden and cell states. At every time step, the current input and previous hidden state are combined to compute a set of gate values, which are split into master gates — one forget gate and one input gate per chunk — and the standard per-token gates found in an ordinary LSTM. The master gates are passed through a cumulative-softmax function so that the forget gate rises monotonically and the input gate falls monotonically across the chunk order, and their element-wise product defines an overlap region in which the standard gates are blended with the master gates to produce the effective forget and input gates. These effective gates then update the memory cell and hidden state in the usual LSTM manner, except that for any padded time step the previous hidden and cell states are simply carried forward unchanged. The hidden states produced by one layer, after dropout, become the input to the next memory layer, and the output sequence of the final layer is returned as the decoded representations Z.

Sample Output — decoded representation Z and master gates

Review	**Z[CLS] — 1×5 row vector**	**Master forget gate (truncated)**	**Master input gate (truncated)**
A	[-0.2756, -0.2404, 0.0918, -0.0643, 0.0745]	0.02, 0.12, 0.44, 0.46, 0.84, 0.88, 0.93, …, 0.99, 1.00	0.99, 0.98, 0.97, …, 0.81, 0.60, 0.19, 0.00
B	[-0.2909, -0.3105, 0.2442, 0.2059, 0.3493]	0.91, 0.92, 0.92, 0.94, …, 0.99, 1.00	0.99, 0.98, 0.97, …, 0.79, 0.75, 0.00
C	[-0.3221, -0.2834, 0.2028, 0.0481, 0.0627]	0.22, 0.83, 0.90, 0.91, …, 0.99, 1.00	0.99, 0.98, 0.95, …, 0.80, 0.74, 0.00

Algorithm 3 ON-LSTM Decoder

Input: Encoder output H = $[h_1, \dots, h_n]$, attention mask M, hidden size d_h, chunk size C, number of memory layers K

Output: Decoded token representations Z = $[z_1, \dots, z_n]$

$n_{chunk} = \frac{d_h}{C}$ // number of neuron chunks

U = H // input of the first memory layer

for each memory layer k = 1 to K

Initialise hidden state $s_0 = 0$ and cell state $c_0 = 0$

for each time step t = 1 to n

gates = $W_{ih}\ U_t + W_{hh}\ s_{t-1}$ + b

// Split gates into master gates ($\tilde{f}, \tilde{\imath}$), one per chunk, standard gates ($o, \hat{c}, i, f$), one per token

$\tilde{f}$ = cumax($\tilde{f}$); $\tilde{\imath}$ = 1 − cumax($\tilde{\imath}$) // cumax(x)k = $\Sigma(j \leq k)$ $softmax(x)_j$

$\omega = \tilde{f} \circ \tilde{\imath}$ // overlap of master gates

$f = f \circ \omega + (\tilde{f} - \omega)$; $i = i \circ \omega + (\tilde{\imath} - \omega)$

$c_t = f \circ c_t - 1 + i \circ \hat{c}$ // hierarchical memory update

$s_t = o \circ \tanh(c_t)$

if $M_t = 0$ then keep previous s, c and output o // padding

U = Dropout(outputs s_1 ... s_n) // input of next layer

Z = output sequence of layer K

Return Z

5.5 ATTENTION POOLING AND CLASSIFIER

Each decoded token representation from Module 4 is passed through a learned linear projection followed by a tanh activation to produce an intermediate vector, which is then reduced to a single importance score for that token via a learned weight vector. Any token at a padded position has its score forced to negative infinity so that it receives no weight. A softmax is applied across all the scores in the sequence to obtain a normalised attention-weight distribution α, and the final review-level vector s is computed as the weighted sum of the decoded token representations using these attention

weights, compressing the whole variable-length sequence into one fixed-size vector while keeping track of which tokens contributed most to it.

Algorithm 4 Attention Pooling

Input: Decoded representations Z = $[z_1, \dots, z_n]$, attention mask M
Output: Review-level vector s, attention weights $\alpha = [\alpha_1, \dots, \alpha_n]$
for each token t = 1 to n
$u_t = tanh(W_a\, z_t + b_a)$ // projection
$e_t = v^t\, u_t$ // importance score of the token
if $M_t = 0$ then $e_t \leftarrow -\infty$ // padding gets no attention
α = softmax$(e_1, \dots, e_n)$
$s = \Sigma\, \alpha_t\, z_t$, for t = 1 to n
Return s, α

Sample Output — attention weights α over the token sequence

Review	Attention weights (selected tokens)
A	[CLS] 0.007, There 0.036, …, on 0.063, his 0.062, …, but 0.063, he 0.063, is 0.063, not 0.062, …, movie 0.060, [SEP] 0.026
B	[SEP] 0.067, [CLS] 0.059, . 0.053, story 0.053, me 0.053 (remaining tokens 0.043–0.053)
C	[CLS] 0.020, What 0.018, …, . 0.074, The 0.075, plot 0.076, was 0.075, boring 0.076, , 0.076, see 0.076, [SEP] 0.060

5.6 ADVERSARIAL TRAINING MODULE

For each mini-batch, a normal forward pass computes the cross-entropy loss on the clean input, and this loss is back-propagated to obtain gradients with respect to the model parameters as well as the gradient of the loss with respect to the word-embedding matrix itself. A copy of the original embedding matrix is saved as a backup, and provided the gradient norm is non-zero and finite, a small perturbation is computed by scaling the normalised embedding gradient by a perturbation strength ε and added directly to the embedding matrix. A second forward pass is then run on these perturbed embeddings to compute an adversarial loss, which is back-propagated so its gradients accumulate on top of those from the clean pass. The embedding matrix is then restored to its original, unperturbed values, the combined gradient norm is clipped to a maximum value for

stability, and the parameters are updated using the AdamW optimiser together with a linear warm-up learning-rate schedule.

Sample Output — three mini-batch examples

Example	Gradient norm	Perturbation size	Clean loss	Adversarial loss
1	0.558	0.6922	0.0798	1.8086
2	0.558	0.0009	0.0060	0.0063
3	0.558	0.0014	0.0023	0.0055

Algorithm 5 Adversarial Training (Fast Gradient Method on Embeddings)

Input: Model parameters θ, mini-batch (X, M, y), perturbation strength ϵ, clip value g_{max}

Output: Updated parameters θ

for each mini-batch

L = CrossEntropy($f(X, M; \theta), y$) // clean loss

Back-propagate L to obtain gradients $\nabla\, \theta$ L

E = word-embedding matrix of the DeBERTa encoder; $g_E = \nabla$ E L

Save backup E_{orig} = E

if $\|g_E\|_2 \neq 0$ and $\|g_E\|_2$is finite then

$r_{adv} = \frac{\varepsilon \cdot g_E}{\|g_E\|_2}$ // adversarial perturbation

E = E + r_{adv}

L_{adv} = CrossEntropy($f(X, M; \theta), y$) // loss on perturbed embeddings

// Back-propagate L_{adv} (gradients are added to those of L)

Restore E = E_{orig}

// Clip the gradient norm to g_{max} & Update θ with AdamW and the linear warm-up scheduler

Return θ

5.7 HYPER-PARAMETER OPTIMIZATION MODULE

Table 5.3 Search space of the Optuna study

Hyper-parameter	Range / choices	Distribution
Encoder learning rate	1×10^{-5} – 5×10^{-5}	Log-uniform
Head learning rate	1×10^{-4} – 2×10^{-3}	Log-uniform
Weight decay	0 – 0.1	Uniform
Warm-up ratio	0 – 0.15	Uniform
Dropout	0.05 – 0.3	Uniform
ON-LSTM hidden size	128, 256, 384	Categorical
Chunk size	8, 16, 32	Categorical
FGM strength ε	0.1 – 1.5	Uniform
Number of ON-LSTM layers	1, 2	Categorical
Batch size	16, 32	Categorical

Sample Output — trial history and selected hyperparameters

Trial	F1 Score
0	0.901
1	0.909
2	0.914
3	Pruned
4	0.918
5	0.920

A search space is defined over the learning rates, weight decay, warm-up ratio, dropout rate, ON-LSTM hidden size, chunk size and adversarial perturbation strength, and an empty trial history is initialised. For the first few trials, hyperparameter values are drawn randomly (log-uniformly for the learning rates) to seed the search; for every trial after that, the TPE splits the trial history into a smaller group of the best-performing trials and a larger group of the rest, fits a probability density to each group, draws a batch of candidate configurations from the good-group density, and selects the candidate that maximises the ratio of the good-group to bad-group density as the next configuration to

try. For each chosen configuration, a model is built and trained epoch by epoch on the tuning subset using the adversarial training procedure of Module 3; after every epoch, precision, recall and F1-score are computed on the validation subset, and the F1-score is reported back to the study. If, after a minimum number of trials and epochs, a trial's F1-score falls below the median score of previous trials at the same epoch, it is pruned and stopped early, with its last reported score kept as its final score. Once all trials have either completed or been pruned, the trial with the highest recorded score is selected as the best configuration, which is then used to train the final model on the complete training set. The pooled review vector from Module 6 is first passed through a dropout layer for regularisation, then through a hidden linear layer followed by a GELU activation, and through a second dropout layer. A final linear layer maps this hidden representation down to two logits, one for each sentiment class, and a softmax converts these logits into class probabilities. During training, the cross-entropy loss is computed between the predicted probabilities and the true label to drive learning, while at inference time the predicted class is simply taken as whichever of the two probabilities is higher, giving the final positive-or-negative sentiment prediction for the review.

Sample Output — logits, softmax scores and final prediction

Example	Logits — [Neg, Pos]	Softmax scores [Neg, Pos]	Predicted class
1	[-2.2639, 2.8433]	[0.0060, 0.9940]	Positive
2	[1.1345, -1.3534]	[0.9233, 0.0767]	Negative
3	[2.8741, -3.2081]	[0.9977, 0.0023]	Negative

Algorithm 6 Optuna Hyperparameter Controller

Input: Search space Λ = {$lr \in [2 \times 10^{-5}, 5 \times 10^{-5}]$, $lr_{head} \in [10^{-4}, 6 \times 10^{-4}]$, $wd \in [0, 0.1]$, $\rho_{warm} \in [0.06, 0.2]$, $p \in [0.05, 0.3]$, $d_h \in \{128, 256\}$, $C \in \{8, 16, 32\}$, $\varepsilon \in [0.1, 1.0]$}, number of trials N_{trials}, training subset D_s, validation subset D_v, maximum epochs E, random-start trials $N_0 = 4$, pruner trials $N_p = 3$

Output: Best hyperparameter set λ^* = argmax *score*

Initialise trial history $H = \emptyset$

for each trial n = 1 to N_{trials}

if $n \leq N_0$ then

$\lambda_n \sim$ Uniform(Λ) // random start (log-uniform for the learning rates)

else // TPE: Suggest Algo

k = min($\lceil 0.1\,|H| \rceil, 25$)

H_g = k trials of H with the highest score

$H_b = \frac{H}{H_g}$

$l(\lambda)$ = Parzen density estimate of H_g;

$g(\lambda)$ = Parzen density estimate of H_b

// Draw candidates $\lambda^1, \ldots, \lambda^K \sim l(\lambda)$, with K = 24

$\lambda_n = argmax_k \left(\frac{l(\lambda^k)}{(g(\lambda^k))}\right)$ //maximises the expected improvement

for each epoch e = 1 to E //Build the hybrid model with λ_n

Train the model on D_s with adversarial training (Algorithm 3)

$P = \frac{TP}{TP + FP}$

$R = \frac{TP}{TP + FN}$ on D_v

$\mathrm{F1}_e = \frac{2 \cdot P \cdot R}{P + R}$ // objective function value

Report $\mathrm{F1_e}$ to the study

if $n > N_p$ and $e > 1$ and $\mathrm{F1_e} < \mathrm{median}\{\mathrm{F1}e(m) : m = 1 \ldots n-1\}$ then // Median pruner: Prune Algo

Mark trial n as pruned and stop it (its last $F1_e$ is kept as its score)

$score_n = \max_e \ \mathrm{F1}_e$

$H = H \cup \{(\lambda_n, score_n)\}$

$\lambda* = \lambda_n$ with the highest $score_n$, $n = 1 \ldots N_{trials}$

//Save λ* and the table of all trials; train the final model on the full training set with λ* (Algorithm 3)

Return λ*

5.10 SUMMARY

The chapter described the modular implementation of the data pipeline, the proposed model, the training engine with adversarial training, the Optuna tuning script, the final training and prediction scripts, and the automated tests. The next chapter presents the results obtained with this implementation.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 OVERVIEW

This chapter discusses about the experimental results of the proposed algorithms compared with various state of art algorithms including the dataset description, optuna search results, its convergence search with graphical representations and tabular representation.

6.2 DATASET DESCRIPTION

The experiments use the 50,000 reviews of the IMDB corpus, which are exactly balanced between the positive and negative classes. After the stratified split described in Section 4.3.1 the training set has 35,000 reviews, the validation set 5,000 reviews and the test set 10,000 reviews, with an equal number of positive and negative reviews in each part (Figure 6.1). The validation set was used for early stopping, model selection and hyper-parameter search, whereas the test set was used only for the final evaluation. Because the classes are balanced, accuracy and F1-score are both meaningful indicators and the values are not distorted by a majority class.

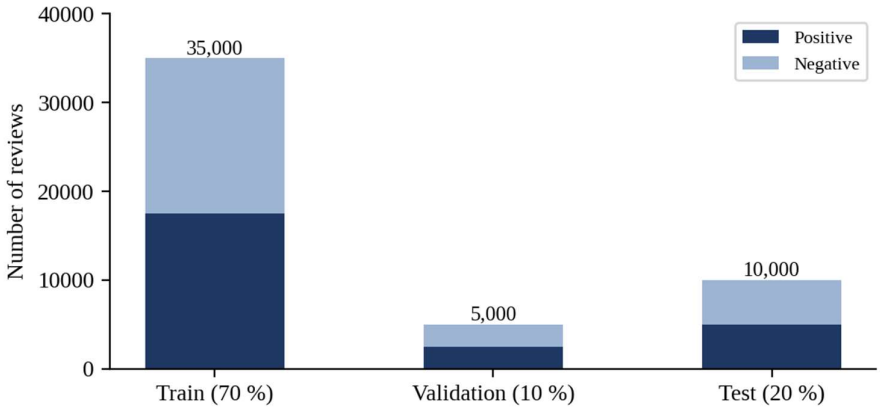


Figure 6.1 Class distribution of the training, validation and test sets

6.3 OPTUNA HYPER-PARAMETER SEARCH RESULTS

The Optuna study used trials of up to six epochs on the 10 % tuning subset (3,500 training and 500 validation reviews; 219 mini-batches of 16 reviews per epoch), using the search configuration of Section 4.8.2. Six trials were run: four ran to completion and two (trials 3 and 5) were pruned by the median pruner, after the third and the second epoch respectively. Completed trials took about 41 to 42 minutes each (about 6.9

minutes per epoch), whereas the pruned trials stopped after about 14 and 22 minutes. Table 6.2 gives the validation F1-score of every trial after each epoch, Table 6.3 the sampled learning rates, weight decay, warm-up ratio and dropout of the trials whose parameters were logged, and Figure 6.2 plots the curves.

Table 6.2 Optuna trials: validation F1-score after each epoch

Trial	State	Hidden / chunk / layers	ε	E1	E2	E3	E4	E5	E6
0	Complete	384 / 16 / 2	1.265	0.9314	0.9363	0.9347	0.9336	0.9383	0.9436
1 (best)	Complete	128 / 32 / 1	0.820	0.9296	0.9322	0.9426	0.9244	0.9347	0.9458
2	Complete	128 / 8 / 1	0.148	0.9345	0.9234	0.9319	0.9356	0.9412	0.9392
3	Pruned	n/l	n/l	0.9244	0.9330	0.9293	–	–	–
4	Complete	384 / 16 / 1	0.108	0.9251	0.9395	0.9286	0.9417	0.9402	0.9436
5	Pruned	n/l	n/l	0.8730	0.9314	–	–	–	–

E1 to E6: epochs; n/l: not logged (the parameters of pruned trials were not printed in the console log).

Table 6.3 Sampled parameters of the logged trials

Trial	Encoder learning rate	Head learning rate	Weight decay	Warm-up ratio	Dropout
0	1.83×10^{-5}	1.73×10^{-3}	0.0732	0.0898	0.089
1	1.34×10^{-5}	2.49×10^{-4}	0.0525	0.0648	0.123
2	2.66×10^{-5}	1.67×10^{-4}	0.0065	0.1423	0.291
4	1.87×10^{-5}	2.25×10^{-4}	0.0829	0.0535	0.120

Trial 1 had the highest score (0.9458, reached at epoch 6) and was selected; trials 0 and 4 (0.9436) and trial 2 (0.9412) were close behind. The scores of the completed trials differ by at most 0.0046, which corresponds to about two to three reviews of the 500-review validation subset, so the ranking among them should be interpreted with caution. The curves are not monotonic (trial 1 drops to 0.9244 at epoch 4 and recovers to 0.9458 at epoch 6), which is why the best epoch over the whole run, rather than the score after a fixed number of epochs, is used as the score of a trial. No trial collapsed to a trivial classifier: the lowest first-epoch F1-score was 0.8730 (trial 5), and the logged trials all used encoder learning rates between 1.34×10^{-5} and 2.66×10^{-5}, in line with the stability guidelines of [3].

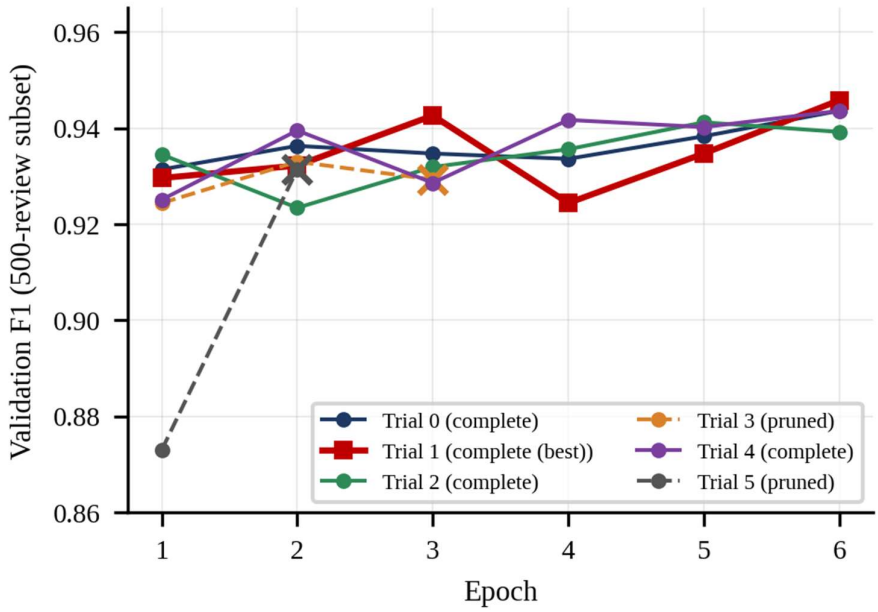


Figure 6.2 Validation F1-score of each Optuna trial after every epoch

6.4 CONVERGENCE OF THE OPTUNA SEARCH

Figure 6.3 shows the optimisation history of the search: the objective value of every trial and the best value found so far. The best-so-far value rose from 0.9436 (trial 0) to 0.9458 (trial 1) and then stayed constant for the remaining four trials: trial 2 and trial 4 did not improve it, and trials 3 and 5 were pruned. The search therefore reached its best value within the first two trials and did not improve afterwards, which is a plateau within the available budget but not a proof of convergence to the global optimum.

Two remarks are needed to interpret this plateau. First, Optuna's TPE sampler uses random sampling for its first start-up trials (ten by default, unless configured otherwise), so with only six trials the suggestions were essentially random and the model-based part of TPE had little opportunity to concentrate on promising regions. Second, the median pruner starts to act only after its three start-up trials, which is consistent with the first pruned trial being trial 3. A larger budget of at least ten to twenty trials would be needed to observe the convergence of the sampler itself.

Figure 6.4 shows the computing time of the trials whose start time was logged. The three completed trials took about 41 to 42 minutes each, while the pruned trials 3 and 5 stopped after about 14 and 22 minutes. Compared with running both of them to the end, pruning saved about 47 minutes, roughly one fifth of the time that five full trials would have needed, without changing the selected configuration.

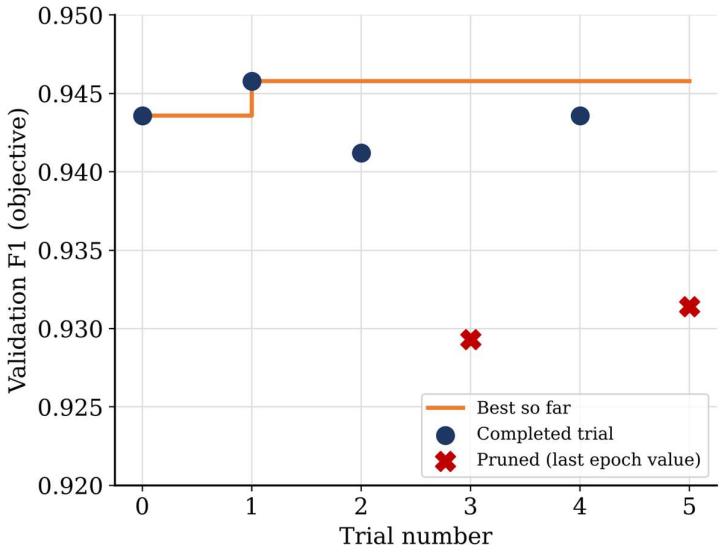


Figure 6.3 Optimisation history (convergence) of the Optuna search

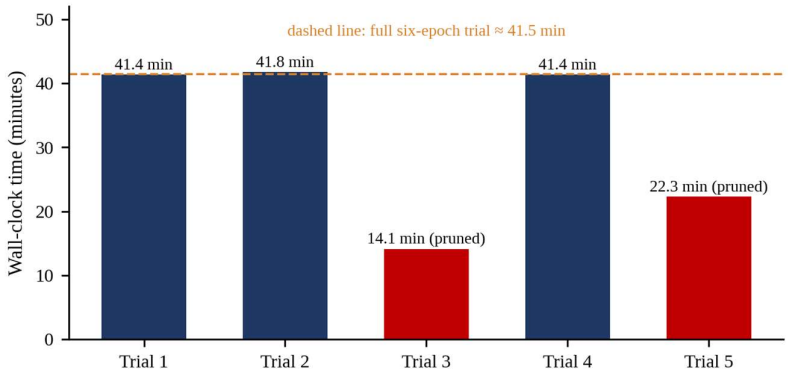


Figure 6.4 Computing time of the Optuna trials (trial 0: start time not logged)

6.5 FINAL MODEL RESULTS

The configuration of trial 1 was used to train the final model on the complete training split of 35,000 reviews (Table 6.4). The maximum sequence length was 256 tokens, one ON-LSTM layer with 128 hidden units in chunks of 32 neurons was used, the FGM strength was 0.820 and the batch size was 16.

Table 6.4 Hyper-parameters of the final model

Hyper-parameter	Value
Encoder	deberta-v3-base
Encoder learning rate	1.34×10^{-5}
Head learning rate	2.49×10^{-4}

Hyper-parameter	Value
Weight decay	0.0525
Warm-up ratio	0.0648
Dropout	0.123
ON-LSTM hidden size / chunk size / layers	128 / 32 / 1
FGM strength ε	0.820
Batch size / maximum length	16 / 256
Epochs (maximum) / patience	6 / 2 (stopped after epoch 4)
Gradient clipping / seed	1.0 / 42

The validation F1-score rose from 0.9474 after the first epoch to a maximum of 0.9562 after the second epoch and then stayed at 0.9557 to 0.9558, while the training loss kept falling (0.2200 to 0.0518) and the validation loss rose (0.1364 to 0.1885), as shown in Table 6.5 and Figure 6.5. Early stopping with a patience of two epochs therefore ended the training after the fourth epoch, and the weights of the second epoch were kept.

Table 6.5 Epoch-wise results of the final training

Epoch	Train loss	Val. accuracy	Val. precision	Val. recall	Val. F1	Val. loss
1	0.2200	0.9466	0.9339	0.9612	0.9474	0.1485
2 (best)	0.1171	0.9560	0.9520	0.9604	0.9562	0.1364
3	0.0788	0.9554	0.9477	0.9640	0.9558	0.1540
4	0.0518	0.9554	0.9498	0.9616	0.9557	0.1885

Table 6.6 Performance of the final model

Split	Accuracy	Precision	Recall	F1-score	Loss
Validation (5,000)	0.9560	0.9520	0.9604	0.9562	0.1364
Test (10,000)	0.9536	0.9531	0.9542	0.9536	0.1408

On the test set the final model reaches an accuracy of 95.36 % and an F1-score of 95.36 %. The test metrics are within 0.26 percentage points of the validation metrics, and the slightly lower test values are expected because the best epoch was selected on the validation set rather than on the test set.

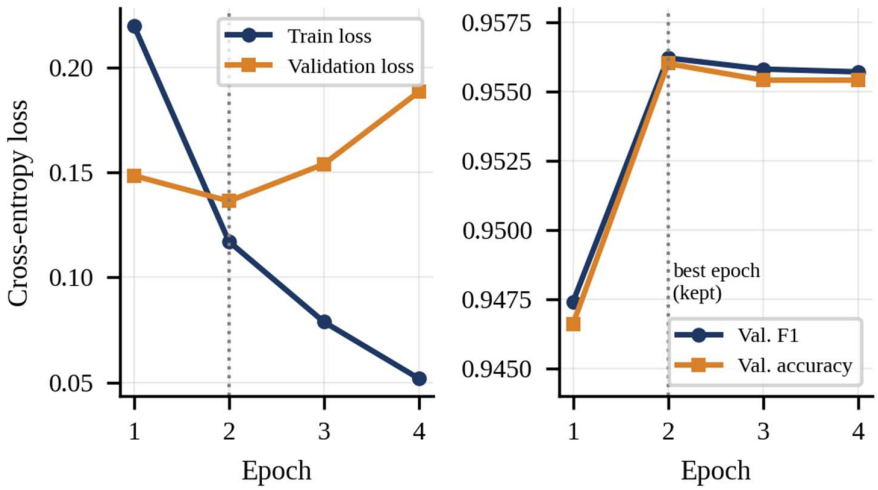


Figure 6.5 Training and validation loss (left), validation F1-score and accuracy (right) of the final model

6.6 ERROR ANALYSIS

Since the test set is balanced, the confusion matrices of the final model can be reconstructed exactly from its accuracy, precision and recall (Table 6.7 and Figure 6.6). On the test set it classifies 4,765 negative and 4,771 positive reviews correctly and makes 235 false-positive and 229 false-negative errors, so 464 of 10,000 reviews (4.64 %) are misclassified. The false-positive rate (4.70 %) and the false-negative rate (4.58 %) are almost equal, so the errors are balanced between the two classes.

Table 6.7 Confusion matrix counts (reconstructed from the reported metrics)

Split	TN	FP	FN	TP
Validation	2,379	121	99	2,401
Test	4,765	235	229	4,771

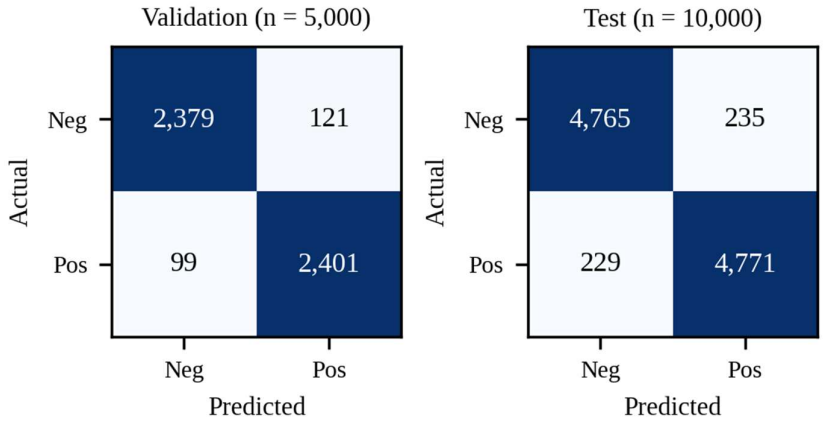


Figure 6.6 Confusion matrices of the final model on the validation and test sets

6.7 COMPARISON WITH THE EXISTING SYSTEM

TRALSem [25] is the most closely related existing system: a Transformer-based sentiment classifier that adds a random-seed strategy, adversarial training (FGM, PGD, FreeLB or YOPO) and a linear output layer, and that was evaluated on IMDB among other datasets. Table 6.8 contrasts the two systems.

Table 6.8 TRALSem versus the proposed model: experimental set-up

Aspect	TRALSem [25]	Proposed Model (this project)
Encoder	BERT-base (uncased), 12 Transformer layers	DeBERTa-v3-base + ON-LSTM decoder
Output head	Linear (regression) layer + softmax	Attention pooling + fully connected layer + softmax
Loss function	Root-mean-square error	Cross-entropy
Robustness	Random seed; FGM, PGD, FreeLB or YOPO	FGM ($\varepsilon = 0.820$)
Tuning	Fixed (4 epochs, learning rate 2×10^{-5} in the baseline comparisons)	Optuna (up to 6 epochs, TPE + median pruner)
Max. length / batch size	256 / 16	256 / 16
IMDB split	40,000 train / 10,000 test	35,000 train / 5,000 validation / 10,000 test
Metrics reported	Macro-/micro-F1, RMSE, precision, recall	Accuracy, precision, recall, F1 (+ macro-averaged)

TRALSem reports a macro-F1, micro-F1, precision and recall of 0.9523 each and an RMSE of 0.1307 on IMDB [25]. To compare like with like, the macro-averaged metrics of the proposed model were computed from the reconstructed test confusion matrix of Table 6.7; on the balanced test set all four equal 0.9536 when rounded (Table

6.9). The RMSE of TRALSem is not comparable because the proposed model uses a classification head with a cross-entropy loss (test loss 0.1408).

Table 6.9 IMDB test metrics: TRALSem versus the proposed model

Metric	TRALSem [25]	Proposed Model	Difference
Macro-F1	0.9523	0.9536	+0.0013
Micro-F1 / accuracy	0.9523	0.9536	+0.0013
Precision (macro)	0.9523	0.9536	+0.0013
Recall (macro)	0.9523	0.9536	+0.0013
RMSE	0.1307	–	–

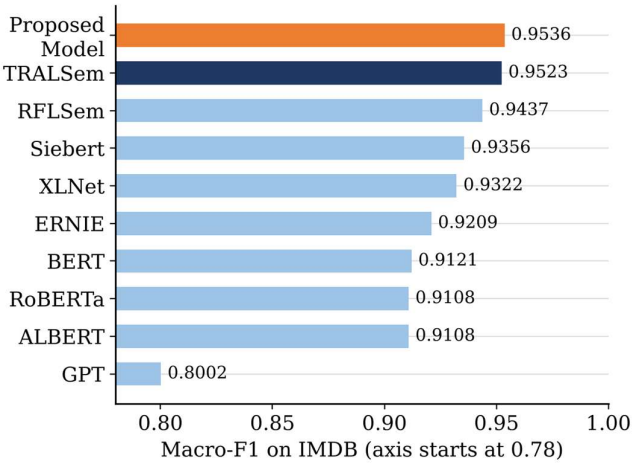


Figure 6.7 IMDB macro-F1 of the proposed model and of the systems reported in [25] (axis truncated at 0.78).

Figure 6.7 places the proposed model among the systems evaluated in Tralsem [25]. Its macro-F1 of 0.9536 is 0.0013 above TRALSem (0.9523), 0.0099 above its predecessor RFLSem (0.9437), 0.0180 above the sentiment-specific Siebert model (0.9356), 0.0214 above the best standard pretrained baseline XLNet (0.9322) and 0.0415 above BERT (0.9121). Figure 6.8 shows that the difference is consistent across precision, recall and F1-score, and Figure 6.9 compares the proposed model with the four adversarial strategies evaluated in [25].

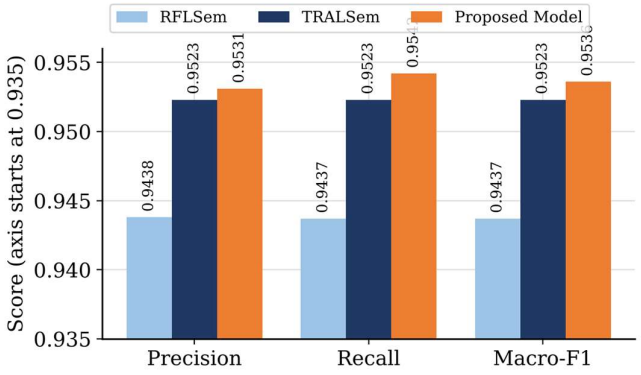


Figure 6.8 Precision, recall and macro-F1 on IMDB: RFLSem and TRALSem as reported in [25] and the proposed model (axis truncated at 0.935)

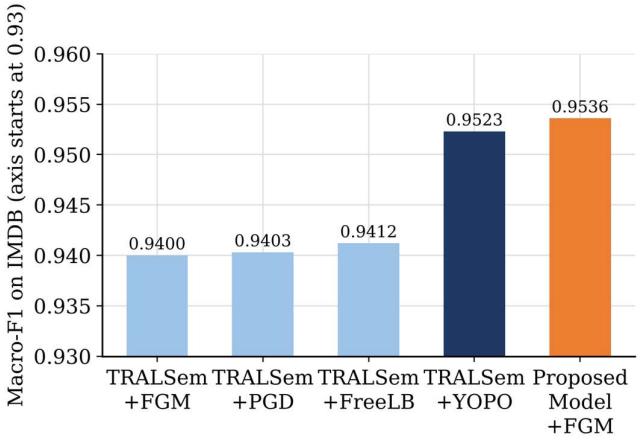


Figure 6.9 IMDB macro-F1 of the TRALSem adversarial variants reported in [25] and of the proposed model with FGM (axis truncated at 0.93)

With the same family of attack (FGM), the proposed model is 0.0136 above the TRALSem-FGM variant (0.9400) and 0.0013 above the best TRALSem variant (YOPO, 0.9523). A possible explanation is that the ON-LSTM decoder, attention pooling and the tuned hyper-parameters, rather than the adversarial method, account for much of the difference, but this was not verified by an ablation.

6.8 SUMMARY

The comparison with TRALSem must be read with the following cautions. (i) The two studies use different random splits of IMDB: TRALSem trains on 40,000 reviews, the proposed model on 35,000. (ii) The TRALSem figures are copied from its published tables and were not re-run in this project. (iii) Both results come from a single configuration, and the margin of 0.13 percentage points corresponds to about 13 of the 10,000 test reviews, which is smaller than the run-to-run variation usually seen when

fine-tuning Transformers [3]. The result therefore shows accuracy comparable to, and not demonstrably better than, the existing system.

Beyond accuracy, the proposed model uses fewer training reviews, tunes its hyper-parameters automatically and exposes token-level attention weights, whereas its sequential ON-LSTM is slower than a plain Transformer head, and TRALSem has been evaluated on Chinese and finer-grained datasets that were not part of this project. The updated Optuna study also has clear limitations: only six trials were run, the tuning subset contains 500 validation reviews, and the parameters of the two pruned trials were not logged. A multi-seed evaluation on identical splits, ablation studies and a re-run of TRALSem under the same protocol are needed to establish whether the small difference is significant.

The Optuna search used trials of up to six epochs, reached its best objective value within the first two trials, and produced a final model with 95.36 % test accuracy and a 95.36 % F1-score, with a validation F1-score of 95.62 %. Compared with the existing TRALSem system, which reports a macro-F1 of 0.9523 on IMDB, the proposed model is marginally higher under a different split, so the two systems should be regarded as comparable until a protocol-matched comparison is made.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

In this study, we implemented and evaluated the proposed model, a sentiment classifier that combines a DeBERTa-v3 encoder, an ON-LSTM decoder, attention pooling and FGM adversarial training, with hyper-parameters selected by Optuna using TPE sampling and median pruning over trials of up to six epochs (SDG 9). On the IMDB dataset, with a stratified 70/20/10 split, the tuned model achieved 95.36 % accuracy, 95.31 % precision, 95.42 % recall and a 95.36 % F1-score on the test set, with a nearly identical 95.62 % validation F1-score, which is comparable to the 95.23 % macro-F1 reported for the existing TRALSem system [25] under a different data split. The study also showed that unstable settings, particularly high learning rates, can be detected and pruned automatically, that a compact recurrent decoder is sufficient on top of a strong pretrained encoder, and that the Optuna search reached its best configuration within the first two of six trials.

7.2 FUTURE ENHANCEMENTS

- Perform a larger Optuna search that also includes the number of ON-LSTM layers, the batch size and the adversarial strength schedule, and use several random seeds per configuration.
- Investigating in diverse languages and domains would significantly increase its practical value and generalizability.
- Addressing complex expressions so that the model could involve incorporating advanced mechanisms into the model architecture or training to capture these nuances.

7.3 SOCIAL IMPACTS AND BENEFITS

- Automatic and explainable analysis of public opinion helps film producers, distributors and small businesses to understand audience reactions, supporting economic productivity and employment in the creative and digital industries (SDG 8).

- Summaries of the opinion of other viewers help consumers to make informed choices about the content and products they use, which supports responsible consumption (SDG 12).
- The efficient search with early pruning shows how computing resources and energy can be saved while developing machine-learning models (SDG 12).
- The documented and tested open-source code gives students and young researchers hands-on experience in natural language processing, deep learning and experiment management (SDG 4).
- The use of openly available datasets and libraries and the sharing of the implementation promote knowledge sharing and collaboration within the research community (SDG 17).
- The hybrid architecture is an example of innovation in language technology that can be extended to other text analytics problems (SDG 9).

7.4 SUMMARY

This chapter concluded the work, proposed future enhancements and described the social impacts of the project, which relate mainly to innovation, education, economic productivity, responsible consumption and partnerships.

APPENDIX 1

WORKING ENVIRONMENT

Hardware Requirement

Processor	:	Intell i5 12th Gen
GPU	:	RTX 3050 / T2 from kaggle
RAM	:	8GB / 32GB from kaggle
Hard Disk	:	512 GB

Software Requirement

Operating System	:	Windows
Programming Language	:	Python 3
Deep Learning Framework	:	PyTorch
Pretrained Encoder	:	microsoft/deberta-v3-base (Hugging Face Transformers)
Hyper-parameter Optimisation	:	Optuna (TPE sampler, median pruner, SQLite storage)
Data and Metrics	:	scikit-learn, pandas, Hugging Face datasets
Dataset	:	IMDB Large Movie Review dataset (50,000 reviews)

Commands to reproduce the experiments

```
python tune.py --n_trials 6 --subset 0.1 --epochs 6
python train.py --params outputs/optuna/best_params.json --epochs 6 --
max_len 128
python predict.py --ckpt outputs/final "There is a smile on his face, but
he is not actually good with movie"
```

The maximum length in best_params.json (256 tokens) overrides the --max_len option of the training script, so the final model was trained with a maximum length of 256 tokens. The dataset was loaded from the stanfordnlp/imdb repository of the Hugging Face hub.

APPENDIX 2

SOURCE CODE

Listing: onlstm.py

```
"""Ordered-Neuron LSTM (Shen et al., 2019, arXiv:1810.09536).

Implements the "ON-LSTM Decoder" block of the system diagram: K stacked
"Memory Layers", each one an ON-LSTM layer that uses the cumax() activation
(Eq. 4 of the ON-LSTM paper) to build master forget / input gates that give
the neurons a hierarchical (high-level = long-term, low-level = short-term)
ordering.
"""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F


def cumsoftmax(x: torch.Tensor, dim: int = -1) -> torch.Tensor:
    """cumax(x)_k = sum_{j<=k} softmax(x)_j"""
    return torch.cumsum(F.softmax(x, dim=dim), dim=dim)


class ONLSTMCell(nn.Module):
    def __init__(self, input_size: int, hidden_size: int, chunk_size: int, dropconnect
        : float = 0.0):
        super().__init__()
        if hidden_size % chunk_size != 0:
            raise ValueError("hidden_size must be divisible by chunk_size")
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.chunk_size = chunk_size
        self.n_chunk = hidden_size // chunk_size

        # 4 standard gates (per neuron) + 2 master gates (per chunk)
        out = self.n_chunk * 2 + hidden_size * 4
        self.ih = nn.Linear(input_size, out)
        self.hh = nn.Linear(hidden_size, out)
        self.drop_weight = nn.Dropout(dropconnect) if dropconnect > 0 else nn.Identity
            ()

    def init_state(self, batch: int, device, dtype):
        h = torch.zeros(batch, self.hidden_size, device=device, dtype=dtype)
        c = torch.zeros(batch, self.n_chunk, self.chunk_size, device=device, dtype=dty
            pe)
        return h, c

    def forward(self, x, state):
        hx, cx = state
        gates = self.ih(x) + self.hh(self.drop_weight(hx))

        cin, cforget = gates[:, : self.n_chunk * 2].chunk(2, 1)
        rest = gates[:, self.n_chunk * 2 :].reshape(-1, self.n_chunk * 4, self.chunk_s
            ize)
        outgate, cell, ingate, forgetgate = rest.chunk(4, 1)

        # master gates (ordered neurons)
        cin = (1.0 - cumsoftmax(cin)).unsqueeze(-1)  # master input gate (descending)
        cforget = cumsoftmax(cforget).unsqueeze(-1)  # master forget gate (ascending)

        ingate = torch.sigmoid(ingate)
        forgetgate = torch.sigmoid(forgetgate)
        cell = torch.tanh(cell)
        outgate = torch.sigmoid(outgate)
```

```
        overlap = cforget * cin
        forgetgate = forgetgate * overlap + (cforget - overlap)
        ingate = ingate * overlap + (cin - overlap)

        cy = forgetgate * cx + ingate * cell
        hy = outgate * torch.tanh(cy)
        return hy.reshape(-1, self.hidden_size), cy


class ONLSTMLayer(nn.Module):
    """One ON-LSTM "Memory Layer" unrolled over time with padding-mask support."""

    def __init__(self, input_size, hidden_size, chunk_size, dropconnect=0.0):
        super().__init__()
        self.cell = ONLSTMCell(input_size, hidden_size, chunk_size, dropconnect)

    def forward(self, x: torch.Tensor, mask: torch.Tensor):
        # x: (B, T, D)  mask: (B, T) with 1 for real tokens
        B, T, _ = x.shape
        h, c = self.cell.init_state(B, x.device, x.dtype)
        outs = []
        for t in range(T):
            h_new, c_new = self.cell(x[:, t], (h, c))
            m = mask[:, t].to(x.dtype).unsqueeze(1)
            h = m * h_new + (1 - m) * h  # freeze state on padding
            c = m.unsqueeze(-1) * c_new + (1 - m.unsqueeze(-1)) * c
            outs.append(h_new * m)
        return torch.stack(outs, dim=1)  # (B, T, H)


class ONLSTMDecoder(nn.Module):
    """K stacked memory layers (Memory Layer 1 ... Memory Layer K)."""

    def __init__(self, input_size, hidden_size=256, chunk_size=16, num_layers=1, dropo
        ut=0.1, dropconnect=0.0):
        super().__init__()
        sizes = [input_size] + [hidden_size] * num_layers
        self.layers = nn.ModuleList(
            [ONLSTMLayer(sizes[i], sizes[i + 1], chunk_size, dropconnect) for i in ran
            ge(num_layers)]
        )
        self.drop = nn.Dropout(dropout)
        self.out_dim = hidden_size

    def forward(self, x, mask):
        for i, layer in enumerate(self.layers):
            x = layer(x, mask)
            if i < len(self.layers) - 1:
                x = self.drop(x)
        return x
```

Listing: model.py

```
"""Hybrid model: DeBERTa encoder -> ON-LSTM decoder -> Attention pooling -> FC -> Soft
    max."""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F
from transformers import AutoModel, AutoConfig, AutoModelForSequenceClassification  #
    noqa: F401

from .onlstm import ONLSTMDecoder


class AttentionPooling(nn.Module):
    """s = sum_t alpha_t z_t, alpha = softmax(v^T tanh(W z_t))  (Eq. 5)."""
```

```
    def __init__(self, dim: int, attn_dim: int | None = None):
        super().__init__()
        attn_dim = attn_dim or dim
        self.proj = nn.Linear(dim, attn_dim)
        self.v = nn.Linear(attn_dim, 1, bias=False)

    def forward(self, z, mask):
        scores = self.v(torch.tanh(self.proj(z))).squeeze(-1)  # (B, T)
        scores = scores.float().masked_fill(mask == 0, float("-inf"))
        alpha = torch.softmax(scores, dim=-1).to(z.dtype)
        s = torch.bmm(alpha.unsqueeze(1), z).squeeze(1)  # (B, D)
        return s, alpha


class ProposedModel(nn.Module):
    def __init__(
        self,
        encoder_name: str = "microsoft/deberta-v3-base",
        num_labels: int = 2,
        onlstm_hidden: int = 256,
        chunk_size: int = 16,
        onlstm_layers: int = 1,
        dropout: float = 0.1,
        encoder_config=None,
    ):
        super().__init__()
        if encoder_config is not None:  # random-init (used by tests, no download)
            self.encoder = AutoModel.from_config(encoder_config)
        else:
            self.encoder = AutoModel.from_pretrained(encoder_name)
            self.encoder.float()
        d = self.encoder.config.hidden_size
        self.enc_drop = nn.Dropout(dropout)
        self.decoder = ONLSTMDecoder(d, onlstm_hidden, chunk_size, onlstm_layers, drop
            out)
        self.pool = AttentionPooling(onlstm_hidden)
        # Fully connected layer (hidden + output), followed by softmax
        self.classifier = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(onlstm_hidden, onlstm_hidden // 2),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(onlstm_hidden // 2, num_labels),
        )

    def head_parameters(self):
        for m in (self.decoder, self.pool, self.classifier):
            yield from m.parameters()

    def forward(self, input_ids, attention_mask, labels=None, return_attention=False):
        H = self.encoder(input_ids=input_ids, attention_mask=attention_mask).last_hidd
            en_state
        Z = self.decoder(self.enc_drop(H), attention_mask)
        s, alpha = self.pool(Z, attention_mask)
        logits = self.classifier(s)  # softmax applied in the loss / predict_proba
        out = {"logits": logits}
        if labels is not None:
            out["loss"] = F.cross_entropy(logits.float(), labels)
        if return_attention:
            out["attention"] = alpha
        return out

    @torch.no_grad()
    def predict_proba(self, input_ids, attention_mask):
        return torch.softmax(self.forward(input_ids, attention_mask)["logits"].float()
            , dim=-1)
```

Listing: adversarial.py

```
"""Adversarial training (FGM - Fast Gradient Method on the word embeddings).
```

```
Corresponds to the "Adversarial Training" block attached to the DeBERTa encoder
in the system diagram. Each step: normal forward/backward -> perturb the
embedding matrix along the gradient direction -> second forward/backward on the
perturbed input (gradients accumulate) -> restore the embeddings.
"""
import torch



class FGM:
    def __init__(self, model, epsilon: float = 1.0, emb_name: str = "word_embeddings")
        :
        self.model = model
        self.epsilon = epsilon
        self.emb_name = emb_name
        self.backup = {}


    def attack(self):
        for name, p in self.model.named_parameters():
            if p.requires_grad and self.emb_name in name and p.grad is not None:
                self.backup[name] = p.data.clone()
                norm = torch.norm(p.grad)
                if torch.isfinite(norm) and norm != 0:
                    p.data.add_(self.epsilon * p.grad / norm)


    def restore(self):
        for name, p in self.model.named_parameters():
            if name in self.backup:
                p.data = self.backup[name]
        self.backup = {}
```

Listing: engine.py

```
"""Training / evaluation engine shared by train.py and tune.py."""
from __future__ import annotations

import copy
import random
from dataclasses import dataclass, asdict


import numpy as np
import torch
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from torch.utils.data import DataLoader
from transformers import get_linear_schedule_with_warmup

from .adversarial import FGM
from .model import ProposedModel



@dataclass
class HParams:
    encoder_name: str = "microsoft/deberta-v3-base"
    lr: float = 2e-5              # encoder learning rate
    head_lr: float = 1e-3         # ON-LSTM / pooling / FC learning rate
    weight_decay: float = 0.01
    warmup_ratio: float = 0.06
    dropout: float = 0.1
    onlstm_hidden: int = 256
    chunk_size: int = 16
    onlstm_layers: int = 1
    adv_epsilon: float = 1.0      # 0 disables adversarial training
    batch_size: int = 32
    max_len: int = 256
    epochs: int = 5
    patience: int = 2
    grad_clip: float = 1.0
    seed: int = 42


    def to_dict(self):
        return asdict(self)
```

```
def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)



def get_device():
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")



def build_model(hp: HParams, encoder_config=None):
    return ProposedModel(
        encoder_name=hp.encoder_name,
        onlstm_hidden=hp.onlstm_hidden,
        chunk_size=hp.chunk_size,
        onlstm_layers=hp.onlstm_layers,
        dropout=hp.dropout,
        encoder_config=encoder_config,
    )



def compute_metrics(y_true, y_pred):
    p, r, f1, _ = precision_recall_fscore_support(y_true, y_pred, average="binary", ze
        ro_division=0)
    return {"accuracy": accuracy_score(y_true, y_pred), "precision": p, "recall": r, "
        f1": f1}



@torch.no_grad()
def evaluate(model, loader, device, amp=True):
    model.eval()
    ys, ps, loss_sum, n = [], [], 0.0, 0
    for b in loader:
        b = {k: v.to(device) for k, v in b.items()}
        with torch.autocast(device.type, dtype=torch.float16, enabled=amp and device.t
            ype == "cuda"):
            out = model(b["input_ids"], b["attention_mask"], b["labels"])
        loss_sum += out["loss"].item() * len(b["labels"])
        n += len(b["labels"])
        ps += out["logits"].argmax(-1).tolist()
        ys += b["labels"].tolist()
    m = compute_metrics(ys, ps)
    m["loss"] = loss_sum / max(n, 1)
    return m



def fit(hp: HParams, train_ds, val_ds, collate, trial=None, encoder_config=None, log=p
    rint, amp=True):
    """Train with FGM adversarial training; returns (best_val_metrics, best_state_dict
        )."""
    set_seed(hp.seed)
    device = get_device()
    model = build_model(hp, encoder_config).to(device)
    train_loader = DataLoader(train_ds, batch_size=hp.batch_size, shuffle=True, collat
        e_fn=collate, num_workers=2)
    val_loader = DataLoader(val_ds, batch_size=hp.batch_size * 2, shuffle=False, colla
        te_fn=collate, num_workers=2)


    no_decay = ("bias", "LayerNorm.weight", "layernorm", "norm")
    head_ids = {id(p) for p in model.head_parameters()}
    groups = []
    for lr, is_head in ((hp.lr, False), (hp.head_lr, True)):
        for decay in (True, False):
            params = [
                p for n, p in model.named_parameters()
                if (id(p) in head_ids) == is_head and (not any(nd in n for nd in no_de
```

```
            cay)) == decay
            ]
            if params:
                groups.append({"params": params, "lr": lr, "weight_decay": hp.weight_d
            ecay if decay else 0.0})
    optim = torch.optim.AdamW(groups)
    total = len(train_loader) * hp.epochs
    sched = get_linear_schedule_with_warmup(optim, int(hp.warmup_ratio * total), total
        )

    use_amp = amp and device.type == "cuda"
    scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
    fgm = FGM(model, epsilon=hp.adv_epsilon) if hp.adv_epsilon > 0 else None

    best, best_state, bad = None, None, 0
    for epoch in range(hp.epochs):
        model.train()
        run = 0.0
        for step, b in enumerate(train_loader):
            b = {k: v.to(device) for k, v in b.items()}
            with torch.autocast(device.type, dtype=torch.float16, enabled=use_amp):
                loss = model(b["input_ids"], b["attention_mask"], b["labels"])["loss"]
            scaler.scale(loss).backward()

            if fgm is not None:  # adversarial step
                fgm.attack()
                with torch.autocast(device.type, dtype=torch.float16, enabled=use_amp)
            :
                    loss_adv = model(b["input_ids"], b["attention_mask"], b["labels"])
            ["loss"]
                scaler.scale(loss_adv).backward()
                fgm.restore()

            scaler.unscale_(optim)
            torch.nn.utils.clip_grad_norm_(model.parameters(), hp.grad_clip)
            scaler.step(optim)
            scaler.update()
            sched.step()
            optim.zero_grad(set_to_none=True)
            run += loss.item()
            if step % 100 == 0:
                log(f"epoch {epoch+1} step {step}/{len(train_loader)} loss {loss.item(
            ):.4f}")

        m = evaluate(model, val_loader, device, amp)
        log(f"[epoch {epoch+1}] train_loss={run/len(train_loader):.4f} val: " +
            " ".join(f"{k}={v:.4f}" for k, v in m.items()))

        if trial is not None:  # Optuna: report intermediate value, allow pruning
            import optuna
            trial.report(m["f1"], epoch)
            if trial.should_prune():
                raise optuna.TrialPruned()

        if best is None or m["f1"] > best["f1"]:
            best, bad = m, 0
            best_state = copy.deepcopy({k: v.detach().cpu() for k, v in model.state_di
            ct().items()}) \
                if trial is None else None
        else:
            bad += 1
            if bad >= hp.patience:
                log("early stopping")
                break
    return best, best_state
```

Listing: data.py

```
"""IMDB Movie Reviews: 50,000 reviews, 2 classes (Positive / Negative).

Split (matches the "Dataset Details" slide): 70 % train (35,000) /
20 % test (10,000) / 10 % validation (5,000), stratified.
```

```
"""
from __future__ import annotations

import html
import re
import unicodedata

import torch
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset

_TAG = re.compile(r"<[^>]+>")
_URL = re.compile(r"https?://\S+|www\.\S+")
_WS = re.compile(r"\s+")


def clean_text(t: str) -> str:
    """Algorithm 1 - text cleaning: strip HTML/URLs, NFKC-normalise, squeeze whitespac
        e."""
    t = html.unescape(t)
    t = _TAG.sub(" ", t)
    t = _URL.sub(" ", t)
    t = unicodedata.normalize("NFKC", t)
    return _WS.sub(" ", t).strip()


def load_imdb_splits(seed: int = 42, csv_path: str | None = None):
    """Returns dict(train/val/test -> (texts, labels)).

    Uses HuggingFace `datasets` ("imdb"), or a CSV with columns `review,sentiment`
    (the Kaggle 'IMDB Dataset of 50K Movie Reviews') if csv_path is given.
    """
    if csv_path:
        import pandas as pd

        df = pd.read_csv(csv_path)
        texts = df["review"].tolist()
        labels = (df["sentiment"].str.lower() == "positive").astype(int).tolist()
    else:
        from datasets import load_dataset

        ds = load_dataset("imdb")
        texts = list(ds["train"]["text"]) + list(ds["test"]["text"])
        labels = list(ds["train"]["label"]) + list(ds["test"]["label"])

    texts = [clean_text(t) for t in texts]
    x_tr, x_tmp, y_tr, y_tmp = train_test_split(texts, labels, test_size=0.30, stratif
        y=labels, random_state=seed)
    x_te, x_va, y_te, y_va = train_test_split(x_tmp, y_tmp, test_size=1 / 3, stratify=
        y_tmp, random_state=seed)
    return {"train": (x_tr, y_tr), "val": (x_va, y_va), "test": (x_te, y_te)}


class EncodedDataset(Dataset):
    """Pre-tokenised with the DeBERTa tokenizer ([CLS] ... [SEP])."""

    def __init__(self, texts, labels, tokenizer, max_len=256):
        enc = tokenizer(texts, truncation=True, max_length=max_len)
        self.ids = enc["input_ids"]
        self.labels = list(labels)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, i):
        return self.ids[i], self.labels[i]
```

```
def make_collate(pad_id: int):
    def collate(batch):
        L = max(len(ids) for ids, _ in batch)
        input_ids = torch.full((len(batch), L), pad_id, dtype=torch.long)
        mask = torch.zeros((len(batch), L), dtype=torch.long)
        for i, (ids, _) in enumerate(batch):
            input_ids[i, : len(ids)] = torch.tensor(ids)
            mask[i, : len(ids)] = 1
        labels = torch.tensor([y for _, y in batch], dtype=torch.long)
        return {"input_ids": input_ids, "attention_mask": mask, "labels": labels}

    return collate
```

Listing: train.py

```
"""Final training on the full 70/20/10 split (optionally with Optuna's best params), t
    hen test evaluation.

  python train.py                                    # defaults
  python train.py --params outputs/optuna/best_params.json
"""
import argparse
import json
import os


import torch
from torch.utils.data import DataLoader
from transformers import AutoTokenizer


from ondeberta.data import EncodedDataset, load_imdb_splits, make_collate
from ondeberta.engine import HParams, build_model, evaluate, fit, get_device



def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--params", default=None, help="best_params.json from tune.py")
    ap.add_argument("--encoder", default="microsoft/deberta-v3-base")
    ap.add_argument("--csv", default=None)
    ap.add_argument("--epochs", type=int, default=5)
    ap.add_argument("--max_len", type=int, default=256)
    ap.add_argument("--adv_epsilon", type=float, default=None, help="0 disables advers
        arial training")
    ap.add_argument("--out", default="outputs/final")
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()
    os.makedirs(args.out, exist_ok=True)


    hp = HParams(encoder_name=args.encoder, epochs=args.epochs, max_len=args.max_len,
        seed=args.seed)
    if args.params:
        for k, v in json.load(open(args.params)).items():
            setattr(hp, k, v)
        hp.epochs = args.epochs
    if args.adv_epsilon is not None:
        hp.adv_epsilon = args.adv_epsilon


    tok = AutoTokenizer.from_pretrained(hp.encoder_name)
    s = load_imdb_splits(args.seed, args.csv)
    print({k: len(v[0]) for k, v in s.items()})
    ds = {k: EncodedDataset(*v, tok, hp.max_len) for k, v in s.items()}
    collate = make_collate(tok.pad_token_id)


    best_val, state = fit(hp, ds["train"], ds["val"], collate)
    print("best val:", best_val)


    device = get_device()
    model = build_model(hp).to(device)
    model.load_state_dict(state)
    test = evaluate(model, DataLoader(ds["test"], batch_size=hp.batch_size * 2, collat
```

```
        e_fn=collate), device)
    print("TEST:", {k: round(v, 4) for k, v in test.items()})

    torch.save({"state_dict": state, "hparams": hp.to_dict()}, f"{args.out}/model.pt")
    tok.save_pretrained(f"{args.out}/tokenizer")
    json.dump({"hparams": hp.to_dict(), "val": best_val, "test": test}, open(f"{args.o
        ut}/results.json", "w"), indent=2)


if __name__ == "__main__":
    main()
```

Listing: predict.py

```
"""Predict sentiment for a review and show the attention-pooling weight of each token.

  python predict.py --ckpt outputs/final "There is a smile on his face, but he is not
      actually good with movie"
"""
import argparse

import torch
from transformers import AutoTokenizer

from ondeberta.data import clean_text
from ondeberta.engine import HParams, build_model, get_device


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--ckpt", default="outputs/final")
    ap.add_argument("text")
    a = ap.parse_args()

    ck = torch.load(f"{a.ckpt}/model.pt", map_location="cpu")
    hp = HParams(**ck["hparams"])
    tok = AutoTokenizer.from_pretrained(f"{a.ckpt}/tokenizer")
    dev = get_device()
    model = build_model(hp).to(dev)
    model.load_state_dict(ck["state_dict"])
    model.eval()

    enc = tok(clean_text(a.text), truncation=True, max_length=hp.max_len, return_tenso
        rs="pt").to(dev)
    with torch.no_grad():
        out = model(enc["input_ids"], enc["attention_mask"], return_attention=True)
    p = torch.softmax(out["logits"].float(), -1)[0].cpu()
    print(f"Negative: {p[0]:.3f}   Positive: {p[1]:.3f}  ->  {'Positive' if p[1] > p[0
        ] else 'Negative'}")
    toks = tok.convert_ids_to_tokens(enc["input_ids"][0])
    for t, w in zip(toks, out["attention"][0].cpu().tolist()):
        print(f"{t:>15s}  {w:.3f}  {'#' * int(w * 60)}")


if __name__ == "__main__":
    main()
```

Listing: tune.py

```
"""Optuna hyper-parameter optimisation for the proposed model (GPU-ready for Kaggle).

Usage:  python tune.py --n_trials 20 --subset 0.3 --epochs 3
"""
import argparse
import gc
import json
import os

import optuna
```

```
import torch
from transformers import AutoTokenizer

from ondeberta.data import EncodedDataset, load_imdb_splits, make_collate
from ondeberta.engine import HParams, fit



def free_gpu():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()



def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--encoder", default="microsoft/deberta-v3-base")
    ap.add_argument("--csv", default=None, help="optional local IMDB csv (review,senti
        ment)")
    ap.add_argument("--n_trials", type=int, default=20)
    ap.add_argument("--epochs", type=int, default=3, help="max epochs per trial")
    ap.add_argument("--subset", type=float, default=0.3, help="fraction of train/val u
        sed per trial")
    ap.add_argument("--max_len", type=int, default=256)
    ap.add_argument("--out", default="/kaggle/working/outputs/optuna")
    ap.add_argument("--study_name", default="ondeberta")
    ap.add_argument("--seed", type=int, default=42)
    args = ap.parse_args()
    os.makedirs(args.out, exist_ok=True)


    if not torch.cuda.is_available():
        raise SystemExit("No GPU found. In Kaggle: Settings > Accelerator > GPU T4 x2
            / P100, then rerun.")
    print("Using GPU:", torch.cuda.get_device_name(0), "| GPUs visible:", torch.cuda.d
        evice_count())


    tok = AutoTokenizer.from_pretrained(args.encoder)
    splits = load_imdb_splits(args.seed, args.csv)


    def sub(xy, frac):
        x, y = xy
        n = max(64, int(len(x) * frac))
        return x[:n], y[:n]  # splits are already shuffled/stratified


    tr = EncodedDataset(*sub(splits["train"], args.subset), tok, args.max_len)
    va = EncodedDataset(*sub(splits["val"], args.subset), tok, args.max_len)
    collate = make_collate(tok.pad_token_id)


    def objective(trial: optuna.Trial) -> float:
        hp = HParams(
            encoder_name=args.encoder,
            lr=trial.suggest_float("lr", 1e-5, 5e-5, log=True),
            head_lr=trial.suggest_float("head_lr", 1e-4, 2e-3, log=True),
            weight_decay=trial.suggest_float("weight_decay", 0.0, 0.1),
            warmup_ratio=trial.suggest_float("warmup_ratio", 0.0, 0.15),
            dropout=trial.suggest_float("dropout", 0.05, 0.3),
            onlstm_hidden=trial.suggest_categorical("onlstm_hidden", [128, 256, 384]),
            chunk_size=trial.suggest_categorical("chunk_size", [8, 16, 32]),
            onlstm_layers=trial.suggest_int("onlstm_layers", 1, 2),
            adv_epsilon=trial.suggest_float("adv_epsilon", 0.1, 1.5),
            batch_size=trial.suggest_categorical("batch_size", [16, 32]),
            max_len=args.max_len,
            epochs=args.epochs,
            patience=args.epochs,
            seed=args.seed,
        )
        free_gpu()  # clean leftovers from the previous trial
        oom = False
        try:
            best, _ = fit(hp, tr, va, collate, trial=trial)
        except torch.cuda.OutOfMemoryError:
            oom = True
```

```
        free_gpu()
        if oom:
            print(f"Trial {trial.number}: CUDA out of memory, pruning.")
            raise optuna.TrialPruned()
        return best["f1"]

    study = optuna.create_study(
        study_name=args.study_name,
        direction="maximize",
        sampler=optuna.samplers.TPESampler(seed=args.seed),
        pruner=optuna.pruners.MedianPruner(n_startup_trials=3, n_warmup_steps=1),
        storage=f"sqlite:///{args.out}/study.db",
        load_if_exists=True,
    )
    study.optimize(objective, n_trials=args.n_trials)

    best = {"encoder_name": args.encoder, "max_len": args.max_len, **study.best_params
        }
    with open(f"{args.out}/best_params.json", "w") as f:
        json.dump(best, f, indent=2)
    study.trials_dataframe().to_csv(f"{args.out}/trials.csv", index=False)
    print("Best val F1:", study.best_value)
    print("Best params:", json.dumps(best, indent=2))
    print(f"Saved to {args.out}/best_params.json  -> now run: python train.py --params
         {args.out}/best_params.json")


if __name__ == "__main__":
    main()
```

REFERENCES

[1] FPang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 79–86.

[2] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL), 142–150.

[3] Mosbach, M., Andriushchenko, M., & Klakow, D. (2021). On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. International Conference on Learning Representations (ICLR).

[4] Stanford AI Lab. Large Movie Review Dataset. https://ai.stanford.edu/~amaas/data/sentiment/

[5] Kim, Y. (2014). Convolutional neural networks for sentence classification. Proceedings of EMNLP, 1746–1751.

[6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT, 4171–4186.

[7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems (NeurIPS), 30, 5998–6008.

[8] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with disentangled attention. International Conference on Learning Representations (ICLR).

[9] He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. arXiv:2111.09543.

[10] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

[11] Shen, Y., Tan, S., Sordoni, A., & Courville, A. (2019). Ordered neurons: Integrating tree structures into recurrent neural networks. International Conference on Learning Representations (ICLR). arXiv:1810.09536.

[12] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. International Conference on Learning Representations (ICLR).

[13] Miyato, T., Dai, A. M., & Goodfellow, I. (2017). Adversarial training methods for semi-supervised text classification. International Conference on Learning Representations (ICLR).

[14] Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. Advances in Neural Information Processing Systems (NeurIPS), 24, 2546–2554.

[15] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2623–2631.

[16] Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A novel bandit-based approach to hyperparameter optimization. Journal of Machine Learning Research, 18(185), 1–52.

[17] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. International Conference on Learning Representations (ICLR).

[18] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. International Conference on Learning Representations (ICLR).

[19] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., et al. (2018). Mixed precision training. International Conference on Learning Representations (ICLR).

[20] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS), 32, 8024–8035.

[21] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. Proceedings of EMNLP: System Demonstrations, 38–45.

[22] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[23] Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.

[24] Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers.

[25] Yang, B., Dang, J., Liu, H., & Jin, Z. (2025). TRALSem: A robust model for textual sentiment analysis. IEEE Transactions on Computational Social Systems, 12(6), 4727–4743. https://doi.org/10.1109/TCSS.2025.3558622

[26] Manning, C. D., Surdeanu, M., Bauer, J., Finkel, J. R., Bethard, S., & McClosky, D. (2014). The Stanford CoreNLP natural language processing toolkit. Proceedings of ACL: System Demonstrations, 55–60.

[27] Thelwall, M., Buckley, K., Paltoglou, G., Cai, D., & Kappas, A. (2010). Sentiment strength detection in short informal text. Journal of the American Society for Information Science and Technology, 61(12), 2544–2558.

[28] Baccianella, S., Esuli, A., & Sebastiani, F. (2010). SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining. Proceedings of the 7th International Conference on Language Resources and Evaluation (LREC).

[29] Wang, S., & Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. Proceedings of ACL (Short Papers), 90–94.

[30] Karimi, Z. (2023). Manifold-based sparse representation for opinion mining. Scientific Reports, 13, Article 15904.

[31] Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2003). A neural probabilistic language model. Journal of Machine Learning Research, 3, 1137–1155.

[32] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems (NeurIPS), 26, 3111–3119.

[33] Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. Proceedings of EMNLP, 1532–1543.

[34] Le, Q., & Mikolov, T. (2014). Distributed representations of sentences and documents. Proceedings of ICML, 1188–1196.

[35] Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017). Bag of tricks for efficient text classification. Proceedings of EACL (Short Papers), 427–431.

[36] Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A., & Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. Proceedings of EMNLP, 1631–1642.

[37] Zhang, L., Wang, S., & Liu, B. (2018). Deep learning for sentiment analysis: A survey. WIREs Data Mining and Knowledge Discovery, 8(4), e1253.

[38] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. Advances in Neural Information Processing Systems (NeurIPS), 27, 3104–3112.

[39] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. Proceedings of EMNLP, 1724–1734.

[40] Dai, A. M., & Le, Q. V. (2015). Semi-supervised sequence learning. Advances in Neural Information Processing Systems (NeurIPS), 28, 3079–3087.

[41] Tang, D., Qin, B., & Liu, T. (2015). Document modeling with gated recurrent neural network for sentiment classification. Proceedings of EMNLP, 1422–1432.

[42] Yang, Z., Yang, D., Dyer, C., He, X., Smola, A., & Hovy, E. (2016). Hierarchical attention networks for document classification. Proceedings of NAACL-HLT, 1480–1489.

[43] Lin, Z., Feng, M., dos Santos, C. N., Yu, M., Xiang, B., Zhou, B., & Bengio, Y. (2017). A structured self-attentive sentence embedding. International Conference on Learning Representations (ICLR).

[44] Wang, Y., Huang, M., Zhu, X., & Zhao, L. (2016). Attention-based LSTM for aspect-level sentiment classification. Proceedings of EMNLP, 606–615.

[45] Shen, Y., Lin, Z., Huang, C.-W., & Courville, A. (2018). Neural language modeling by jointly learning syntax and lexicon. International Conference on Learning Representations (ICLR).

[46] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. Proceedings of NAACL-HLT, 2227–2237.

[47] Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI.

[48] Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. Proceedings of ACL, 328–339.

[49] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692.

[50] Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & Le, Q. V. (2019). XLNet: Generalized autoregressive pretraining for language understanding. Advances in Neural Information Processing Systems (NeurIPS), 32.

[51] Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2020). ALBERT: A lite BERT for self-supervised learning of language representations. International Conference on Learning Representations (ICLR).

[52] Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training text encoders as discriminators rather than generators. International Conference on Learning Representations (ICLR).

[53] Zhang, Z., Han, X., Liu, Z., Jiang, X., Sun, M., & Liu, Q. (2019). ERNIE: Enhanced language representation with informative entities. Proceedings of ACL, 1441–1451.

[54] Sun, Y., Wang, S., Li, Y., Feng, S., Tian, H., Wu, H., & Wang, H. (2020). ERNIE 2.0: A continual pre-training framework for language understanding. Proceedings of AAAI, 34(5), 8968–8975.

[55] Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. Proceedings of ICML, PMLR 97, 2790–2799.

[56] Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to fine-tune BERT for text classification? Chinese Computational Linguistics (CCL), Lecture Notes in Computer Science, 11856, 194–206.

[57] Zhang, T., Wu, F., Katiyar, A., Weinberger, K. Q., & Artzi, Y. (2021). Revisiting few-sample BERT fine-tuning. International Conference on Learning Representations (ICLR).

[58] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR).

[59] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. International Conference on Learning Representations (ICLR).

[60] Zhu, C., Cheng, Y., Gan, Z., Sun, S., Goldstein, T., & Liu, J. (2020). FreeLB: Enhanced adversarial training for natural language understanding. International Conference on Learning Representations (ICLR).

[61] Zhang, D., Zhang, T., Lu, Y., Zhu, Z., & Dong, B. (2019). You only propagate once: Accelerating adversarial training via maximal principle. Advances in Neural Information Processing Systems (NeurIPS), 32, 227–238.

[62] Jiang, H., He, P., Chen, W., Liu, X., Gao, J., & Zhao, T. (2020). SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization. Proceedings of ACL, 2177–2190.

[63] Jin, D., Jin, Z., Zhou, J. T., & Szolovits, P. (2020). Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. Proceedings of AAAI, 34(5), 8018–8025.

[64] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305.